cap-able

Make it more able to caption — now that is cap-able

v0.1.0 2026-05-10 MIT

A comprehensive Typst package for creating professional three-line tables and figures with bilingual caption support, continued tables/figures, subfigures, and flexible customization options for academic documents.

Schrödinger Blume

cap-able is a powerful Typst package designed for academic documents, providing:

- **Three-line tables** — Academic-standard tables (top, middle, bottom rules)
- **Bilingual captions** — Auto-formatted bilingual captions (Chinese/English, German/English, etc.)
- **Continued tables/figures** — Multi-page tables and figures with automatic numbering
- **Subfigures** — Flexible multi-image grid layouts with overlay labels and subcaptions
- **Notes** — Auto-width-matched notes for tables and figures
- **Multilingual support** — 25+ languages including RTL languages
- **Unified configuration** — Configure both via `cap-style`, or override per-type with `captab-style` / `capfig-style`

The package is particularly suited for academic papers, theses, and technical documents requiring professional typography and multi-language support.

Table of Contents

Introduction 4
I.1 Design Philosophy 4
I.2 Dependencies 4
Installation 5
II.1 From Typst Universe 5
II.2 Manual Installation 5
Quick Start 6
III.1 Basic Three-Line Table 6
III.2 Basic Figure 6
III.3 Bilingual Captions 7
Tables in Detail 8
IV.1 Table Syntax 8
IV.1.1 Basic Syntax 8
IV.1.2 Alignment 8
IV.1.3 Cell Merging 9
IV.2 Column Configuration 9
IV.2.1 **Auto mode `columns: ()`** 9

IV.2.2 Absolute units `columns: (8cm, 3cm, 2cm)` 10
IV.2.3 fr units `columns: (3fr, 1fr, 1fr)` — First column third as wide 10
IV.2.4 Mixed `columns: (5fr, 3cm, 1fr)` — Value column fixed at 3cm; Description and Unit columns share remaining width at 5:1 10
IV.3 Additional Lines 10
IV.3.1 Global default stroke `extra-rule` 10
IV.4 Three-Line Strokes 11
IV.5 Disable three-line mode (`three-line-table: false`) 12
IV.6 Pass-through `tablem` / `table` advanced usage 12
IV.7 Page Break 13
IV.8 Table Notes 16
IV.9 Continued Tables 16
IV.9.1 Auto page-break + repeat caption 16
IV.9.2 Manual `refer-to` (preserved) . 17
IV.9.3 Continuation Mode 18
Figures in Detail 19
V.1 Single Figures 19
V.2 Subfigures 19
V.3 Overlay Labels 19
V.3.1 Label Style Reference 20
V.3.2 Decoupling overlay vs subcaption: dict form of `label-style` 20
V.3.3 Spacing between subcaption number and body 21
V.4 Subfigure Cross-References 21
V.4.1 `subref-style`: keep `label-style` decorations in `@ref` 22
Configuration 23
VI.1 Unified Configuration 23
VI.2 Table Configuration 23
VI.3 Figure Configuration 24
VI.4 Table Width 24
VI.5 Custom Numbering 25
VI.5.1 String formats 25
VI.5.2 Function form (advanced) 26
VI.5.3 Tables vs figures: separate or unified 26
VI.5.4 Escape hatch: bypass cap-able entirely 27
VI.6 Fine-grained caption text `caption-text` 27
VI.6.1 Flat form — applies to the whole caption 27
VI.6.2 Nested form — per-part overrides 27
VI.6.3 Common patterns 28
VI.6.4 Properties 28
VI.7 Bilingual Outline 28
Multilingual Support 30
VII.1 Supported Languages 30
VII.2 Simplified vs Traditional Chinese . 31
VII.3 RTL Language Support 31
VII.4 Language-Specific Formatting 32
Complete Usage Reference 33
VIII.1 Public API Overview 33
VIII.2 `set-table-width` Complete Parameters 33
VIII.3 `cap-style` Complete Parameters .33
VIII.4 `captab-style` Complete Parameters 35
VIII.5 `capfig-style` Complete Parameters 37
VIII.6 `captab` Complete Parameters 40
VIII.6.1 Continuation Mode Examples 42
VIII.7 `bicap` Standalone Caption Function 43
VIII.7.1 Caption horizontal alignment `caption-align` 43
VIII.7.2 Floating placement `placement` 44
VIII.8 `capfig` Complete Parameters 48
VIII.9 `capsubfig` Complete Parameters .48
VIII.9.1 Subfigure Dict Structure 48
VIII.9.2 `label-style-override` dict keys 49
VIII.9.3 Function Parameters 49
VIII.9.4 Label Style Parse Rules 49

VIII.9.5 Overlay with Circle Background 50
VIII.9.6 Multi-row Subfigures 50
VIII.10 `captnote` Complete Parameters . 50
VIII.11 `capfnote` Complete Parameters . 51
VIII.12 Full Multilingual Rules 51
API Reference 53
IX.1 Configuration Module (config.typ) 53
IX.1.1 cap-style 53
IX.1.2 capfig-style 54
IX.1.3 captab-style 56
IX.1.4 resolve-lang-indent 58
IX.1.5 set-figure-width 58
IX.1.6 set-table-width 58
IX.2 Caption Module (bicap.typ) 58
IX.2.1 bicap 58
IX.3 Table Module (table.typ) 63
IX.3.1 captab 63
IX.4 Note Module (note.typ) 65
IX.4.1 capfnote 65
IX.4.2 captnote 65
IX.5 Figure Module (figure.typ) 65
IX.5.1 capfig 66
IX.5.2 capsubfig 66
Frequently Asked Questions 69
X.1 Why Markdown syntax for tables? . 69
X.2 How do I create tables without captions? 69
X.3 Can I use regular Typst table syntax? 69
X.4 How do I reference tables and figures? 70
X.5 Why doesn't my continued table show the caption? 70
X.6 How do I fix missing indentation after tables in Chinese? 70
X.7 How do I show bilingual captions in the table of contents? 71
X.8 Why no `capsubtab` (sub-tables)? . . . 71
Changelog 73
XI.1 Version 0.1.0 73
XI.2 Version 0.0.2 76
XI.3 Version 0.0.1 76
Index 78

Part I

Introduction

cap-able brings professional table and figure capabilities to Typst, following academic publishing standards and best practices.

The name “cap-able” is a wordplay on “caption” and “able” — making your documents more able to handle complex captions, very capable, huh!

I.1 Design Philosophy

The package follows these design principles:

1. Declarative Configuration: Configure once at the document start, use everywhere
2. Sensible Defaults: Works out of the box with minimal configuration
3. Language Awareness: Automatic formatting based on document language
4. Academic Standards: Follows established conventions for scholarly publishing

I.2 Dependencies

This package depends on:

- `@preview/tablem:0.3.0` — Markdown-style table syntax support

For documentation generation only:

- `@preview/tidy:0.4.3` — Auto-generate API reference from docstrings
- `@preview/mantys:1.0.2` — Documentation template

Part II

Installation

II.1 From Typst Universe

Once published to Typst Universe, import directly:

```
#import "@preview/cap-able:0.1.0": *
```

II.2 Manual Installation

After downloading from the repository:

1. Place the `cap-able` folder in your project directory
2. Import with a relative path:

```
#import "cap-able/0.1.0/lib.typ": *
```

Part III

Quick Start

This chapter provides a quick overview of the most common use cases.

III.1 Basic Three-Line Table

The simplest way to create a table uses Markdown-like syntax:

```
#captab(
  caption: [Sample Data],
)[
  | Name   | Age | Score |
  | ------ | --- | ----- |
  | Alice  | 25  | 95    |
  | Bob    | 30  | 87    |
]
```

Table 1: Sample Data

Name	Age	Score
Alice	25	95
Bob	30	87

The table automatically gets:

- Content-sized width (default `width: auto`; switch to fixed length / ratio via the `width` parameter or `set-table-width(...)` — see “Table Width” section)
- 1.5pt top rule
- 0.5pt rule below the header
- 1.5pt bottom rule
- Centered content with proper spacing

III.2 Basic Figure

Creating figures is equally straightforward:

```
#capfig(
  rect(width: 15%, height: 1.15cm, fill: blue.lighten(80%)),
  caption: [A Simple Rectangle],
)
```

Figure 1: A Simple Rectangle

III.3 Bilingual Captions

For documents requiring bilingual captions, simply add the `caption-en` parameter:

```
#set text(lang: "es") // Set the document language to non-English

#captab(
  caption: [Parámetros experimentales],
  caption-en: [Experimental Parameters],
)[
  | Parámetro   | Valor | Unidad |
  | ----------- | ----- | ------ |
  | Temperatura |  25   |   °C   |
  | Presión     |   1   |   atm  |
]
```

Tabla 2. Parámetros experimentales
Table 2. Experimental Parameters

Parámetro	Valor	Unidad
Temperatura	25	°C
Presión	1	atm

The package automatically:

- Detects the document language
- Formats the primary caption in that language
- Adds an English caption below when `enable-english-caption: true`

Note: When the document language is English (`#set text(lang: "en")`), if both `caption` and `caption-en` are provided, only `caption-en` is displayed as a single-language caption — `caption` is ignored, since the document is already in English and bilingual output is unnecessary.

Part IV

Tables in Detail

IV.1 Table Syntax

`captab` uses the `tablem` package to parse Markdown-style table syntax, avoiding verbose Typst table syntax.

IV.1.1 Basic Syntax

```
| Header1 | Header2 | Header3 |
| ------- | ------- | ------- | <- Separator
| Cell 1  | Cell 2  | Cell 3  |
| Cell 4  | Cell 5  | Cell 6  |
```

Table 3: Basic Syntax Example

Header1	Header2	Header3
Cell 1	Cell 2	Cell 3
Cell 4	Cell 5	Cell 6

Note: The separator row (`| --- |`) marks the boundary between header and body rows.

IV.1.2 Alignment

Control alignment using colons in the separator:

```
| Left      | Center    | Right      |
| :-------- | :-------: | ---------: |
| L         | C         | R          |
```

Table 4: Column Alignment Example

Left	Center	Right
L	C	R

IV.1.3 Cell Merging

tablem supports cell merging:

- Use `<` to merge with cell to the left (horizontal)
- Use `^` to merge with cell above (vertical)

```
| A    | B    |
| ---- | ---- |
| Span | <    |
| C    | Span |
| D    | ^    |
```

Table 5: Cell Merging Example

A	B
Span	
C	Span
D	

IV.2 Column Configuration

Customize column widths using the `columns` parameter (the legacy name `cols` still works as a back-compat alias).

Column width rules:

- Without `columns` (auto mode): columns fit content; may overflow with long text.
- Only absolute units (e.g., `cm`, `pt`): table width = sum of columns; may be narrower or overflow.
- With `fr` units: table expands proportionally to text width. `fr` and absolute units can be mixed — absolute columns take fixed widths first, and the remaining space is distributed by `fr` ratio.

The `columns` parameter accepts an array of lengths, for example:

```
columns: (8cm, 2cm, 2cm)      // Absolute units
columns: (3fr, 1fr, 1fr)      // fr units
columns: (3fr, 2cm, 1fr)      // Mixed
```

IV.2.1 Auto mode `columns: ()`

Table 6: Auto Columns

Description	Value	Unit
This is a very very, really really, truly truly long text	42	m/s

IV.2.2 Absolute units `columns: (8cm, 3cm, 2cm)`

Table 7: Absolute Columns

Description	Value	Unit
This is a very very, really really, truly truly long text	42	m/s

IV.2.3 fr units `columns: (3fr, 1fr, 1fr)` — First column third as wide

Table 8: Proportional Columns

Description	Value	Unit
This is a very very, really really, truly truly long text	42	m/s

IV.2.4 Mixed `columns: (5fr, 3cm, 1fr)` — Value column fixed at 3cm; Description and Unit columns share remaining width at 5:1

Table 9: Mixed Columns

Description	Value	Unit
This is a very very, really really, truly truly long text	42	m/s

IV.3 Additional Lines

Add extra horizontal or vertical lines for complex layouts:

```
#captab(
  hlines: ((row: 3, stroke: 1pt),),   // Add 1pt horizontal line after row 3
  vlines: ((col: 1, start: 1),),      // Add vertical line at column 1 (from row 1)
  caption: [Table with Extra Lines],
)[...]
```

Table 10: Table with Extra Lines

A	B	C
1	2	3
4	5	6
7	8	9

IV.3.1 Global default stroke `extra-rule`

When every extra line in a table (or document) shares the same stroke, set a global default rather than repeating it per line:

```
#show: captab-style.with(extra-rule: 0.5pt + red)

#captab(
  hlines: ((row: 2,), (row: 3,)),     // both inherit 0.5pt + red, no repetition
)[ ... ]
```

`extra-rule` accepts a single value (shared by hlines & vlines) or a dict form for per-axis control:

```
#show: captab-style.with(
  extra-rule: (h: 1pt + blue, v: 0.3pt + gray),
)
```

Per-line `stroke` still wins:

```
#captab(
  extra-rule: 0.5pt + green,
  hlines: (
    (row: 2,),                        // 0.5pt + green ← inherits
    (row: 5, stroke: 1.5pt + orange), // 1.5pt + orange ← overrides
  ),
)
```

Configurable globally on `cap-style` / `captab-style`, or per-call via `captab(extra-rule: ...)`. Default `0.5pt` matches the previous hard-coded behaviour.

IV.4 Three-Line Strokes

Customize the top, middle and bottom rules of the three-line table via `top-rule` / `middle-rule` / `bottom-rule`. Any Typst stroke value works — thickness, color, dashed:

```
#captab(
  caption: [Custom Rule Styles],
  top-rule: 2pt + red,             // 2pt red top rule
  middle-rule: 0.5pt + gray,       // gray middle rule
  bottom-rule: stroke(thickness: 2pt, dash: "dashed"), // dashed bottom
)[
  | A | B | C |
  | - | - | - |
  | 1 | 2 | 3 |
]
```

Table 11: Custom Rule Styles

You can also configure them globally via `captab-style`:

```
#show: captab-style.with(
  top-rule: 1.2pt,
  middle-rule: 0.4pt,
  bottom-rule: 1.2pt,
)
```

`auto` reads from global state (defaults: top/bottom `1.5pt`, middle `0.5pt`).

IV.5 Disable three-line mode (`three-line-table: false`)

By default `captab` renders a three-line table (manual top/middle/bottom rules, table `stroke: none`). To produce a standard Typst grid table (borders on every cell side), set `three-line-table` to `false`:

```
// Per-call
#captab(
  caption: [Standard Typst grid],
  three-line-table: false,
)[
  | ID | Name | Value |
  | -- | ---- | ----- |
  | 1  | A    | 100   |
]

// Globally
#show: captab-style.with(three-line-table: false)
// or
#show: cap-style.with(three-line-table: false)
```

When `three-line-table: false`, `top-rule` / `middle-rule` / `bottom-rule` are inactive; user-defined extra `hlines` / `vlines` still apply. Adjust the default Typst grid stroke globally via `set table(stroke: ...)`.

IV.6 Pass-through `tablem` / `table` advanced usage

Apart from captab's own named parameters (`columns` / `cols` / `align` / `size` / `leading` / `inset` / `caption…` / `…-rule` / `breakable` / `repeat-…` / `hlines` / `vlines` / `label`, etc.), any other named argument is forwarded as-is to the underlying `table(...)` call, mirroring `tablem`'s advanced-usage surface. Useful pass-throughs:

- `fill: color | function` (cell background; can be `(x, y)` callback)

- `stroke: stroke | function | dictionary` (per-cell strokes)
- `gutter` / `column-gutter` / `row-gutter`
- `rows`
- any other named parameter that Typst's `table()` accepts

```
#let frame(stroke) = (x, y) => (
  left: if x > 0 { 0pt } else { stroke },
  right: stroke,
  top: if y < 2 { stroke } else { 0pt },
  bottom: stroke,
)

#captab(
  caption: [Monthly reading list],
  three-line-table: false,                              // full grid instead of three-line
  columns: (0.4fr, 1fr, 1fr),
  align: left,
  fill: (_, y) => if calc.odd(y) { rgb("EAF2F5") },     // alternating row fill
  stroke: frame(rgb("21222C")),                         // custom stroke
)[
  | *Month*  | *Title*               | *Author*           |
  | -------- | --------------------- | ------------------ |
  | January  | The Great Gatsby      | F. Scott Fitzgerald |
  | February | To Kill a Mockingbird | Harper Lee         |
]
```

Table 12: Monthly reading list

Month	Title	Author
January	The Great Gatsby	F. Scott Fitzgerald
February	To Kill a Mockingbird	Harper Lee

IV.7 Page Break

Starting with 0.1.0 `captab` allows long tables to break across pages by default. Two relevant parameters:

- `breakable` (bool, default `true`) — whether the table may break. Set to `false` to keep the table atomic on one page (it may overflow).
- `repeat-header` (bool / int, default `true`) — repeat the header row on each continuation page using Typst's native `table.header(repeat: ...)`. `true` repeats on every continuation page; pass a positive integer `n` to repeat only on the first `n` continuation pages; `false` disables repetition.

- `continued-caption` (bool, default `false`) — whether to render a "Cont. Table X.Y caption" header on each continuation page (same format as `refer-to` mode). The first page still shows the full original caption.

```
#captab(
  caption: [Long Data Table],
  breakable: true,         // allow page break (default)
  repeat-header: true,     // repeat header rows (default)
  continued-caption: true,    // continuation pages show "Cont. Table X.Y"
)[
  | ID | Name | Value |
  | -- | ---- | ----- |
  // ...
]
```

Or configure globally:

```
#show: captab-style.with(
  breakable: true,
  repeat-header: true,
  continued-caption: true,
)
```

Table 13: Long Data Table

ID	Name	Value
001	Apple	12.50
002	Banana	8.30
003	Orange	15.00
004	Grape	22.80
005	Watermelon	35.60
006	Mango	18.90
007	Pineapple	25.40
008	Strawberry	28.70
009	Blueberry	45.20
010	Peach	16.40
011	Pear	10.80
012	Cherry	52.30
013	Pomelo	19.60

Continuation of Table 13: Long Data Table

ID	Name	Value
014	Dragonfruit	21.50
015	Kiwi	14.30
016	Cantaloupe	32.90
017	Coconut	26.80
018	Lychee	38.40
019	Longan	24.70
020	Mangosteen	48.50

How it works: `continued-caption` does not register a new figure on every continuation page. Each continuation cell uses `query(label)` to locate the main table, reads its number from the counter, and delegates to the refer-to branch of `_make_caption_content` to format "Cont. Table X.Y caption …". If no `label` was provided, cap-able auto-synthesises a hidden `__captab_repeat_<n>` label for internal lookup. The caption row and the header row live in two separate `table.header` blocks (with `level: 1/2`), so `continued-caption: true` can coexist with `repeat-header: false` — the caption repeats while the header does not.

The explicit `refer-to` continuation mechanism (manually-split tables) is still supported and useful when you need to place continuation tables at different positions / chapters.

IV.8 Table Notes

Add explanatory notes below tables:

```
#captab(caption: [Statistical Results])[
  | Variable | Mean  | SD   |
  | -------- | ----- | ---- |
  | X        | 3.2\* | 0.5  |
  | Y        | 4.1   | 0.8  |
]
#captnote[
  Note: $\*p < 0.05$. SD = Standard Deviation.
]
```

Table 14: Statistical Results

Variable	Mean	SD
X	3.2*	0.5
Y	4.1	0.8

Note: *$p < 0.05$. SD = Standard Deviation.

IV.9 Continued Tables

cap-able supports two continuation styles — pick the one that fits the use case:

1. Auto page-break + repeat caption (recommended, since 0.1.0) — let `captab` flow naturally across pages and automatically render “Cont. Table X.Y” on the top of every continuation page.
2. Manual `refer-to` (always available) — split the table into two or more parts and use `refer-to` on the second part to inherit the number. Useful when you want to insert text/figures/other logic between the original and the continuation.

IV.9.1 Auto page-break + repeat caption

Pass `continued-caption: true` to `captab` (`breakable` is `true` by default). cap-able uses Typst’s native `table.header(repeat: ...)` to re-emit “Cont. Table X.Y” on the top of every continuation page. The number is anchored to the main table, so no figure is re-registered.

```
#captab(
  caption: [Long Data Table],
  continued-caption: true,         // continuation pages show "Cont. Table X.Y"
  // breakable: true,           // already the default
  // repeat-header: true,       // header also repeats by default
  label: <tab:long-auto>,
)[
```

```
  | ID | Value |
  | -- | ----- |
  | 1  | 100   |
  | 2  | 200   |
  | 3  | 300   |
  // ...
]
```

To repeat only the caption but not the markdown header, pass `repeat-header: false`. To let the table flow without any continuation caption (Typst-default behaviour), keep `continued-caption: false` (the default).

When to use: long data tables, fully-automatic page breaks. One line of `continued-caption: true` does it — no manual splitting, no caption duplication.

Known limitation: `continued-caption: true` is incompatible with `caption-position: bottom`. Continuation repetition relies on `table.header(repeat: true)` to inject the caption inside the table; the only mechanism for the bottom edge would be `table.footer`, but that locks the caption inside the table's column boundaries — visually "embedded in the table" — which contradicts what `caption-position: bottom` means ("outside the table, below it").

When both are set, cap-able silently disables repeat — equivalent to `continued-caption: false`: the original caption appears once, beneath the last page of the table. To repeat the caption on every page, switch to `caption-position: top`.

IV.9.2 Manual `refer-to` (preserved)

When you need to insert explanatory text, footnotes or any other content between the original and the continuation, use `refer-to` to split explicitly:

```
// First part (original table)
#captab(
  caption: [Long Data Table],
  label: <tab:long>,        // set label for continuation reference
)[
  | ID | Value |
  | -- | ----- |
  | 1  | 100   |
]

Arbitrary content can go here (notes, smaller table, image, ⋯).

// Continuation (header shows "Cont. Table X")
```

```
#captab(
  caption: [Long Data Table],
  refer-to: <tab:long>,       // reference the original for numbering
  show-caption: true,         // force the caption text on the continuation
)[
  | ID | Value |
  | -- | ----- |
  | 2  | 200   |
]
```

Table 15: Long Data Table

ID	Value
1	100

Arbitrary content can go here (notes, smaller table, image, ...).

Continuation of Table 15: Long Data Table

ID	Value
2	200

Either way, the continuation automatically:

- Uses the same number as the original table
- Adds the continuation prefix/suffix
- Does not create a new entry in the table of contents

IV.9.3 Continuation Mode

`continued-mode` controls the format (applies to both styles):

- `"prefix"` (default): prefix mode, e.g. “Cont. Table 1”
- `"suffix"`: suffix mode, e.g. “Table 1 (continued)”

Part V

Figures in Detail

V.1 Single Figures

Create single figures with bilingual captions using `capfig`:

```
#capfig(
  image("example.png", width: 30%),
  caption: [Typst Logo],
  label: <fig:typst-logo>,
)
```

V.2 Subfigures

`capsubfig` creates multi-image parallel layouts:

```
#capsubfig(
  (
    (content: rect(width: 3cm, height: 2cm, fill: red.lighten(70%)),
     subcaption: [Red]),
    (content: rect(width: 3cm, height: 2cm, fill: green.lighten(70%)),
     subcaption: [Green]),
    (content: rect(width: 3cm, height: 2cm, fill: blue.lighten(70%)),
     subcaption: [Blue]),
  ),
  columns: 3,
  show-subcaption: true,        // Show subcaptions
  caption: [Color Comparison],
)
```


(a): Red (b): Green (c): Blue

Figure 2: Color Comparison

V.3 Overlay Labels

Use overlay label mode to place (a), (b) labels directly on images for a cleaner look:

```
#capsubfig(
  (
    (content: rect(width: 4cm, height: 3cm, fill: orange.lighten(70%))),
    (content: rect(width: 4cm, height: 3cm, fill: purple.lighten(70%))),
  ),
  columns: 2,
  caption: [Overlay Labels Demo],
  label-mode: "overlay",        // Enable overlay mode
  label-style: "(a)",           // Label style
  label-bg: white.transparentize(20%),  // Semi-transparent white bg
  label-offset: (5pt, 5pt),     // Offset from top-left
)
```

Figure 3: Overlay Labels Demo

V.3.1 Label Style Reference

The `label-style` parameter supports various formats:

Style	Example	Description
`"(a)"`	(a), (b), (c)	Parenthesized lowercase
`"(A)"`	(A), (B), (C)	Parenthesized uppercase
`"(1)"`	(1), (2), (3)	Parenthesized numbers
`"(i)"`	(i), (ii), (iii)	Parenthesized roman lowercase
`"(I)"`	(I), (II), (III)	Parenthesized roman uppercase
`"a)"`	a), b), c)	Trailing parenthesis only
`"Fig. A"`	Fig. A, Fig. B	English prefix

V.3.2 Decoupling overlay vs subcaption: dict form of `label-style`

By default the overlay label stamped on the figure (`label-mode: "overlay"`) and the subcaption number prefix share a single `label-style` — pass a string and both stay in sync.

To control them independently, pass a dictionary:

```
#capsubfig(
  caption: [Compact overlay, full subcaption],
  show-subcaption: true,
  label-mode: "overlay",
  label-style: (overlay: "a)", subcaption: "(a)"),    // overlay "a)", subcaption "(a) caption"
  (
    (content: img1, subcaption: [first sub]),
    (content: img2, subcaption: [second sub]),
  ),
)
```

In the dict, `overlay` controls the overlay style and `subcaption` controls the subcaption prefix; missing keys fall back to the package default `"(a)"`.

Cross-reference letter source: follows whichever side is actually visible — subcaption (when `show-subcaption: true` and `show-subcaption-label: true`) wins, otherwise the overlay style is used (when `label-mode: "overlay"`); when neither is visible the subcaption style is used as a defensive fallback. This keeps the letter in `@fig:xxx` consistent with what the reader actually sees.

V.3.3 Spacing between subcaption number and body

The gap between the subcaption number prefix (e.g. `"(a)"`) and its body text is controlled by `subcaption-number-title-spacing`:

- Default `auto`: inherits the main caption's `number-title-spacing` (the same separator used for "Figure 1.1 caption" in the package defaults), keeping subcaptions visually consistent with the main caption.
- Configure globally via `capfig-style(subcaption-number-title-spacing: ...)`.
- Override per call via `capsubfig(subcaption-number-title-spacing: ...)`.

```
#capsubfig(
  caption: [per-call separator " — "],
  show-subcaption: true,
  label-mode: "overlay",
  subcaption-number-title-spacing: [ — ],
  (...),
)
```

V.4 Subfigure Cross-References

After setting a `label` for each subfigure, reference them with `@`:

```
#capsubfig(
  (
```

```
    (content: rect(fill: gray), label: <fig:sub-a>),
    (content: rect(fill: gray), label: <fig:sub-b>),
  ),
  label-mode: "overlay",
  caption: [Referenced Subfigures],
  label: <fig:main>,
)

See @fig:sub-a and @fig:sub-b for details.
```

Live demonstration:

Figure 4: Referenced Subfigures

See Figure 4a and Figure 4b (overall Figure 4).

V.4.1 `subref-style`: keep `label-style` decorations in `@ref`

By default `subref-style: "letter"` — `@fig:sub-a` renders as `Fig. 1a` (just the letter).

Set `"full"` to also keep the `label-style` decorations in cross-references:

```
#capsubfig(
  ...,
  label-style: "(a)",
  subref-style: "full",        // ← keep the parentheses in @ref
)
```

Renders: `@fig:sub-a` → `Fig. 1(a)`, matching the subcaption prefix.

Works with Chinese-style prefixes (`label-style: "图 a"` + `"full"` → `图 1 图 a`), brackets (`"[A]"` → `Fig. 1[A]`), etc. In `"full"` mode, `label-sep` defaults to empty string since the decorations already separate visually; override `label-sep` explicitly if you want a different separator.

Configurable globally via `capfig-style(subref-style: "full")` or per-call on `capsubfig`.

Part VI

Configuration

VI.1 Unified Configuration

`cap-style` configures shared styles (numbering, caption, language, notes, ...) for both tables and figures at once. A subsequent `captab-style` / `capfig-style` call can override per-type.

```
#show: cap-style.with(
  numbering-format: "1.1",
  use-chapter: true,
  caption-weight: "regular",
  enable-english-caption: true,
)

// Per-type override still works afterwards:
#show: capfig-style.with(
  label-mode: "overlay",
  label-style: "(a)",
)
```

VI.2 Table Configuration

Use `captab-style` to configure all table-related settings. Typically called via `#show:` at the document start:

```
#show: captab-style.with(
  // Numbering
  numbering-format: "1.1",      // Chapter.Number format
  use-chapter: true,            // Include chapter number

  // Caption style
  caption-size: 10.5pt,
  caption-weight: "regular",    // or "bold"

  // Spacing
  caption-above: 1em,           // Above caption
  caption-below: 0.95em,        // Between caption and table

  // Language
  lang: auto,                   // Auto-detect
  enable-english-caption: true, // Enable English sub-caption
```

```
17
18    // Table content
19    body-size: 10.5pt,
20    cell-inset: (x: 3pt, y: 6.5pt),
21
22    // Continuation
23    continued-mode: "prefix",     // "prefix" or "suffix"
24  )
```

VI.3 Figure Configuration

Use `capfig-style` for figure and subfigure settings:

```
1   #show: capfig-style.with(
2     // Numbering
3     numbering-format: "1.1",
4
5     // Subfigure defaults
6     label-mode: "overlay",        // Default overlay mode
7     label-style: "(a)",
8     gutter: 1em,
9     subcaption-pos: "bottom",
10
11    // Caption
12    enable-english-caption: true,
13  )
```

VI.4 Table Width

cap-able does not enforce a fixed table width by default — `width: auto` lets the table size itself by content (a row like `| A | B | C |` only takes three character-widths, not the full text area). The default is decided by `columns`:

- User did not pass `columns`: defaults to `(auto,) * N`, table is content-sized.
- User passed `columns`: respects the user's spec.
  - `(1fr, 1fr, 1fr)` → fr columns expand, but with no fixed-width parent, behaviour depends on context.
  - `(8cm, 4cm)` → table is 12cm wide, centered.

To fix the table width, three options:

```
1   // (1) Per call: captab(width: ...)
2   #captab(caption: [Narrow Table], width: 60%)[ ... ]
3   #captab(caption: [Absolute Width], width: 8cm)[ ... ]
4   #captab(caption: [Decimal Percent], width: 80.5%)[ ... ]
```

```
5
6  // (2) Globally via captab-style; subsequent captab calls follow
7  #show: captab-style.with(width: 70%)
8
9  // (3) Globally via set-table-width; everything after that follows
10 #set-table-width(width: 50%)         // new form
11 #set-table-width(percentage: 50)     // legacy form (1-100 int, converted to ratio)
```

When `width != auto`, if the user did not pass `columns`, cap-able uses `(1fr,) * N` so the table fills the configured width.

`width` accepts three types:

Type	Description / Examples
`auto`	Unconstrained (default). Table sizes to content.
`<length>`	Absolute width. e.g. `8cm`, `120pt`.
`<ratio>`	Percentage of text width, including decimals. e.g. `80%`, `80.5%`.

Priority: per-call `captab(width:)` > global state (set by `captab-style` / `set-table-width`) > default `auto`.

```
1 #show: captab-style.with(width: 70%)
2
3 #captab(caption: [Following the global 70%])[ ... ]                // 70% wide
4 #captab(caption: [Override to 50%], width: 50%)[ ... ]             // 50% wide
```

Table 16: Example: 50% wide

A	B
C	D

VI.5 Custom Numbering

`numbering-format` accepts any value Typst's native `numbering()` would accept — most needs are one-liners.

VI.5.1 String formats

As long as it contains a format placeholder (`1` arabic, `a/A` letter, `i/I` roman), other characters render verbatim:

Format	Renders	Use case
`"1"`	1, 2, 3, ...	Plain arabic (default)
`"(A)"`	(A), (B), (C), ...	Letter + parentheses
`"App. 1"`	App. 1, App. 2, ...	Literal prefix + number

Format	Renders	Use case
`"1.1"`	1.1, 1.2, 2.1, …	Chapter.number (needs `use-chapter: true`)
`"I.A"`	I.A, I.B, II.A, …	Roman chapter + letter index
`"§1-A"`	§1-A, §1-B, …	Symbol + chapter + letter

With `use-chapter: true`, the last placeholder in the format string is treated as the figure/table's own number; the N-1 preceding ones are heading-level prefixes (matching `=`, `==`, `===` …). For example, `"I.1.1.A"` inside `=== 2.3.4` for the 5th table renders as `II.3.4.E`.

VI.5.2 Function form (advanced)

`numbering-format` also accepts a function — equivalent to Typst's native `numbering((..nums) => ..., 1, 2)`:

```
#show: captab-style.with(
  use-chapter: false,
  numbering-format: (..nums) => {
    let n = nums.pos().last()
    [Tab§#n]                    // → "Tab§1", "Tab§2", ...
  },
)
```

With `use-chapter: true`, the function receives all heading levels + the figure/table number (without slicing — your function decides what to consume):

```
#show: captab-style.with(
  use-chapter: true,
  numbering-format: (..nums) => {
    let arr = nums.pos()
    let chap = arr.slice(0, arr.len() - 1)   // chapter chain
    let n = arr.last()                        // figure/table number
    [§#chap.map(str).join(".")—#n]            // "§2.3—5"
  },
)
```

VI.5.3 Tables vs figures: separate or unified

```
// Fully separate (recommended)
#show: captab-style.with(numbering-format: "1.1")    // tables: 1.1, 1.2
#show: capfig-style.with(numbering-format: "I.1")    // figures: I.1, I.2

// Unified via cap-style
```

```
#show: cap-style.with(numbering-format: "1.1")       // both tables & figures: 1.1
```

VI.5.4 Escape hatch: bypass cap-able entirely

cap-able installs `set figure(numbering: ...)` inside `captab-style` / `capfig-style`. If even the function form is not enough, write your own `set figure(...)` after the cap-style call to fully replace it:

```
#show: cap-style.with(...)
#set figure.where(kind: table): set figure(numbering: num => [#sym.section.bold #num])
```

Note: this bypasses cap-able's bilingual supplement / continued-prefix show rules, so confirm you don't need those.

VI.6 Fine-grained caption text `caption-text`

cap-able's caption is not a real `figure.caption` element — to support bilingual layout, the caption is rendered manually as `text(...)` calls. As a result, `show figure.caption: set text(font: ...)` does not work. To change caption font / colour / tracking / etc., use the `caption-text` field.

`caption-text` accepts any parameter Typst's native `text()` takes (`font`, `fill`, `tracking`, `spacing`, `stretch`, ...) and supports both flat and nested forms:

VI.6.1 Flat form — applies to the whole caption

```
#show: cap-style.with(
  caption-text: (font: "Times New Roman", fill: blue),
)
```

The whole caption (prefix + number + body) is wrapped uniformly.

VI.6.2 Nested form — per-part overrides

If the dict contains any of `whole` / `prefix` / `supplement` / `number` / `body`, it is treated as nested:

Layer	Scope
`whole`	Entire caption (outermost default)
`prefix`	Prefix block (supplement + number)
`supplement`	Just the supplement word ("Table" / "Figure")
`number`	Just the digits ("1" / "1.1" / "II.3.4.E" / ...)
`body`	Just the body (the user-supplied caption text)

Layers cascade: inner overrides outer. Example:

```
#show: cap-style.with(
  caption-text: (
    whole:  (font: "Helvetica"),         // outermost default
```

```
    number: (fill: red, weight: "bold"), // overlay: red bold on the digits
    body:   (font: "SimSun"),            // body uses SimSun
  ),
)
```

Result: supplement is Helvetica; number is red bold Helvetica; body is SimSun.

VI.6.3 Common patterns

```
// 1. Whole caption Times
caption-text: (font: "Times New Roman")

// 2. Only the digits red
caption-text: (number: (fill: red))

// 3. Prefix bold Times, body regular SimSun
caption-text: (
  prefix: (font: "Times New Roman", weight: "bold"),
  body:   (font: "SimSun"),
)

// 4. Supplement monospace, number Times, body Helvetica
caption-text: (
  supplement: (font: "Courier"),
  number:     (font: "Times New Roman"),
  body:       (font: "Helvetica"),
)
```

VI.6.4 Properties

- Default (`:`) does not affect rendering — fully backward compatible.
- When a key collides with `caption-size` / `caption-weight`, the dict's value wins (it acts as an override layer).
- Exposed on `cap-style` / `captab-style` / `capfig-style`.
- Applies to continuation captions too (both `continued-caption: true` and the `refer-to` continuation form).

VI.7 Bilingual Outline

By default, only the primary language caption appears in the outline (table of contents). Enable `outline-bilingual` to show both languages in the outline.

```
#show: captab-style.with(
  outline-bilingual: true,       // Enable bilingual outline
  outline-separator: " / ",      // Separator between primary and English
  outline-newline: false,        // false: same line; true: English on new line
```

```
5 )
```

Three related parameters:

- `outline-bilingual`: Show bilingual captions in outline (default `false`)
- `outline-separator`: Separator between primary and English captions (default `" / "`)
- `outline-newline`: Put English caption on a new line (default `false`)

All three configuration functions (`captab-style`, `capfig-style`, `cap-style`) support these parameters. Use `cap-style` to enable bilingual outline for both tables and figures at once.

Part VII

Multilingual Support

The package supports 25+ languages with automatic localization. Just set the document language and the package automatically uses the correct text.

```
#set text(lang: "de")   // Switch to German

#captab(
  caption: [Experimentelle Ergebnisse],
  caption-en: [Experimental Results],
)[...]
// Caption shows: Tabelle 1  Experimentelle Ergebnisse
//                Table 1 Experimental Results
```

VII.1 Supported Languages

Code	Language	Code	Language
en	English	zh	Chinese
de	German	fr	French
es	Spanish	it	Italian
pt	Portuguese	ru	Russian
ja	Japanese	ko	Korean
ar	Arabic ↲	he	Hebrew ↲
fa	Persian ↲	ur	Urdu ↲
nl	Dutch	pl	Polish
cs	Czech	sv	Swedish
da	Danish	no	Norwegian
fi	Finnish	tr	Turkish
el	Greek	hi	Hindi
th	Thai	vi	Vietnamese

↲ = Right-to-left language; Traditional Chinese is distinguished via the `region` parameter.

VII.2 Simplified vs Traditional Chinese

The package distinguishes Simplified Chinese (`zh`) from Traditional Chinese (`zh-TW`) using Typst's `text.region` parameter:

```
// Simplified Chinese (default)
#set text(lang: "zh")

// Traditional Chinese
#set text(lang: "zh", region: "TW")
```

Key differences:

Item	Simplified	Traditional
Table prefix	表	表
Figure prefix	图	圖
Continued table	续表	續表
Continued figure	续图	續圖
Continued suffix	（续）	（續）

表 17　繁體中文測試表格
Table 17 Traditional Chinese Test Table

項目	數值
測試	100

續表 17　繁體中文測試表格
Continuation of Table 17 Traditional Chinese Test Table

項目	數值
測試	100

圖 5　繁體中文測試圖片
Figure 5 Traditional Chinese Test Figure

VII.3 RTL Language Support

For RTL languages (Arabic, Hebrew, Persian, Urdu), the package automatically:

- Reverses bilingual caption line order (RTL first, then English)
- Uses `box[]` wrapping to prevent direction confusion in RTL content

- Forces `dir: ltr` for the English line

جدول 18: أهم المدن في العالم العربي
Table 18: Major Cities in the Arab World

المدينة	الدولة	عدد السكان (مليون)
القاهرة	مصر	20.9
بغداد	العراق	7.5
الرياض	المملكة العربية السعودية	7.2
الخرطوم	السودان	5.8
الدار البيضاء	المغرب	3.7
الجزائر	الجزائر	3.5
دمشق	سوريا	2.5
عمّان	الأردن	4.0
بيروت	لبنان	2.4
تونس	تونس	2.3

VII.4 Language-Specific Formatting

Each language has customized separators and spacing:

- Chinese/Japanese: No space between prefix and number, em-space before title
- French: Space before colon (“Tableau 1 : Title”)
- Spanish/Italian: Dot separator (“Table 1. Title”)
- English/German: Colon+space (“Table 1: Title”)

Part VIII

Complete Usage Reference

This chapter is the authoritative reference listing every public function, every parameter, every accepted value, every alias, and every behavioral detail of capable.

VIII.1 Public API Overview

Function	Category	Purpose
`cap-style`	Config	Shared style for tables & figures
`captab-style`	Config	Global table style
`capfig-style`	Config	Global figure style
`set-table-width`	Config	Global table width %
`captab`	Table	Three-line table
`capfig`	Figure	Single bilingual figure
`capsubfig`	Figure	Subfigure layout
`captnote`	Note	Table note
`capfnote`	Note	Figure note
`bicap`	Caption	Standalone bilingual caption

VIII.2 `set-table-width` Complete Parameters

Param	Type	Description
`width`	`auto` / `length` / `ratio`	New API. `auto` = unconstrained; pass `8cm` / `80%` / `80.5%`
`percentage`	`int` (1–100)	Legacy API (kept for backward compat). Converted to `<int>%`

When both are passed, `width` wins.

```
#set-table-width(width: 80%)        // new form
#set-table-width(width: 8cm)        // absolute width
#set-table-width(width: auto)       // back to unconstrained
#set-table-width(percentage: 80)    // legacy still works
```

VIII.3 `cap-style` Complete Parameters

`cap-style` is the unified entry point: it writes all shared parameters into both the table and figure config states (equivalent to calling `captab-style.with(...)` and `capfig-`

`style.with(...)` in sequence). A subsequent `captab-style` / `capfig-style` call can still override per-type.

Param	Default	Description
`numbering-format`	`"1"`	Numbering format: any string accepted by Typst's native `numbering()` (e.g. `"1.1"`, `"A.1"`, `"§1-A"`) or a function `(..nums) => content`
`use-chapter`	`false`	Include chapter number
`supplement`	`auto`	Main language prefix
`supplement-en`	`auto`	English prefix
`continued-prefix`	`auto`	Continued prefix (main lang)
`continued-prefix-en`	`auto`	Continued prefix (English)
`continued-suffix`	`auto`	Continued suffix (main lang)
`continued-suffix-en`	`auto`	Continued suffix (English)
`continued-mode`	`"prefix"`	Continuation mode `"prefix"` / `"suffix"`
`continued-show-caption`	`auto`	Show caption text in continuation
`caption-size`	`10.5pt`	Caption size
`caption-weight`	`"regular"`	Caption weight
`caption-text`	`(:)`	Pass-through dict for caption `text(...)`. Flat form (e.g. `(font: "Times")`) applies to the whole caption; nested form (any of `whole` / `prefix` / `supplement` / `number` / `body`) layers per part. See "Fine-grained caption text" section.
`caption-leading`	`0.5em`	Caption line spacing
`caption-above`	`1.5em`	Space above caption
`pre-supplement-number-spacing`	`auto`	Prefix–number spacing
`post-supplement-number-spacing`	`auto`	Number–suffix spacing
`number-title-spacing`	`auto`	Number–title separator
`number-title-spacing-en`	`auto`	Number–English-title separator
`lang`	`auto`	Language code override
`enable-english-caption`	`true`	Enable English sub-caption
`outline-bilingual`	`false`	Bilingual outline
`outline-separator`	`" / "`	Outline separator
`outline-newline`	`false`	Newline in outline
`after-indent`	`auto`	Post-block indent fix
`note-above`	`0.7em`	Above note
`note-below`	`1em`	Below note
`note-size`	`10.5pt`	Note size
`note-leading`	`6.5pt`	Note leading

Param	Default	Description
`note-justify`	`true`	Justify note
`caption-position`	`auto`	Caption position in `#bicap()[body]` mode (applies to both table/figure; `auto` keeps each kind's default)
`caption-align`	`auto`	Caption horizontal alignment: string `"center"` / `"left"` / `"right"` / `"text-left"` / `"text-right"`, or dict `(main, continued)` to split
`placement`	`none`	Floating placement: `none` (in-flow) / `top` / `bottom` (float top/bottom of page) / `auto` (Typst picks the closer of top/bottom); forces `breakable: false` when active

Parameters not exposed by `cap-style` — table-only (`body-size` / `body-leading` / `cell-inset` / `inset` / `table-below` / `width` / `three-line-table` / `top-rule` / `middle-rule` / `bottom-rule` / `breakable` / `repeat-header` / `continued-caption`) and figure-only (`figure-above` / `figure-below` / `subcaption-*` / `gutter` / subfigure `label-*`) — must still be configured via `captab-style` / `capfig-style` directly.

VIII.4 `captab-style` Complete Parameters

All parameters of `captab-style` with defaults. `auto` means auto-select based on document language.

Patch semantics: every parameter actually defaults to `auto`, meaning "leave the current state untouched". The "Default" column below shows the value used the first time the function is called. Subsequent `captab-style.with(...)` calls only override the fields you supply — any field you omit keeps its previous value. This lets you patch one dimension at a time without re-stating the entire configuration.

Param	Default	Description
`caption-above`	`1.5em`	Space above caption block
`caption-below`	`0.5em`	Between caption and body
`table-below`	`0.5em`	Below whole table
`caption-leading`	`0.5em`	Caption line spacing
`numbering-format`	`"1"`	Numbering format: any string accepted by Typst's native `numbering()` or a function `(..nums) => content`. See "Custom numbering" section for details.
`use-chapter`	`true`	Include chapter number

Param	Default	Description
`supplement`	`auto`	Main language prefix
`supplement-en`	`auto`	English prefix
`continued-prefix`	`auto`	Continued prefix
`continued-prefix-en`	`auto`	English continued prefix
`continued-suffix`	`auto`	Continued suffix
`continued-suffix-en`	`auto`	English continued suffix
`continued-mode`	`"prefix"`	Continuation mode `"prefix"` / `"suffix"`
`continued-show-caption`	`auto`	Show caption text in continuation
`caption-size`	`10.5pt`	Caption size
`caption-weight`	`"regular"`	Caption weight
`caption-text`	`(:)`	Pass-through dict for caption `text(...)`. Flat form (e.g. `(font: "Times")`) applies to the whole caption; nested form (any of `whole` / `prefix` / `supplement` / `number` / `body`) layers per part. See “Fine-grained caption text” section.
`pre-supplement-number-spacing`	`auto`	Prefix–number spacing
`post-supplement-number-spacing`	`auto`	Number–suffix spacing
`number-title-spacing`	`auto`	Number–title separator
`number-title-spacing-en`	`auto`	Number–English-title separator
`lang`	`auto`	Language override
`enable-english-caption`	`true`	Enable English sub-caption
`body-size`	`10.5pt`	Body font size
`body-leading`	`0.45em`	Body line spacing
`cell-inset`	`(x: 5pt, y: 5pt)`	Cell padding (dict or scalar); alias of `inset`, `cell-inset` wins if both passed
`inset`	`auto`	Alias of `cell-inset` (matches captab’s `inset` parameter name)
`note-above`	`0.5em`	Above note
`note-below`	`1em`	Below note
`note-size`	`10.5pt`	Note size
`note-leading`	`6.5pt`	Note leading
`note-justify`	`true`	Justify note
`outline-bilingual`	`false`	Bilingual outline
`outline-separator`	`" / "`	Outline separator
`outline-newline`	`false`	Newline in outline
`after-indent`	`auto`	Post-table indent fix

Param	Default	Description
width	auto	Table width. auto=unconstrained (content-sized); <length> like 8cm; <ratio> like 80% / 80.5%
three-line-table	true	Three-line table mode (false skips the manual rules and uses Typst's default table() grid stroke)
top-rule	1.5pt	Three-line top rule stroke (any stroke value, e.g. 2pt + red; only used when three-line-table: true)
middle-rule	0.5pt	Three-line middle rule stroke
bottom-rule	1.5pt	Three-line bottom rule stroke
extra-rule	0.5pt	Default stroke for hlines / vlines entries that omit stroke; accepts a single value or (h: ..., v: ...) dict for per-axis control
breakable	true	Allow the table to break across pages (outer block's breakable)
repeat-header	true	Repeat the markdown header row on each continuation page (true / false / positive int n)
continued-caption	false	Repeat the caption on each continuation page using the refer-to ("Cont. Table X.Y") format
caption-position	top	Position of the caption relative to body in #bicap()[body] mode (top / bottom)
caption-align	"center"	Caption horizontal alignment: "center" / "left" / "right" (table-local) / "text-left" / "text-right" (text-area-local); or dict (main, continued)
placement	none	Floating placement: none / top / bottom / auto (forces breakable: false when active)

Spacing parameters may be length or content: any *-spacing or pre/post-supplement-number-spacing accepts a length (e.g. 0.5em) or content directly (e.g. [\u{3000}] em-space).

VIII.5 capfig-style Complete Parameters

capfig-style merges figure style + figure spacing + subfigure defaults, updating all three states at once.

Patch semantics: same as `captab-style` — every parameter defaults to `auto`; only the fields you supply are written back. The "Default" column shows the initial state-dictionary value.

Caption/numbering/language section

Param	Default	Description
`numbering-format`	`"1"`	Numbering format: any string accepted by Typst's native `numbering()` (e.g. `"1.1"`, `"A.1"`, `"§1-A"`) or a function `(..nums) => content`
`use-chapter`	`false`	Include chapter number
`supplement`	`auto`	Main language prefix
`supplement-en`	`auto`	English prefix
`continued-prefix`	`auto`	Continued prefix (main lang)
`continued-prefix-en`	`auto`	Continued prefix (English)
`continued-suffix`	`auto`	Continued suffix (main lang)
`continued-suffix-en`	`auto`	Continued suffix (English)
`continued-mode`	`"prefix"`	Continuation mode `"prefix"` / `"suffix"`
`continued-show-caption`	`auto`	Show caption text in continuation
`caption-size`	`10.5pt`	Caption size
`caption-weight`	`"regular"`	Caption weight
`caption-text`	`(:)`	Pass-through dict for caption `text(...)`. Flat form (e.g. `(font: "Times")`) applies to the whole caption; nested form (any of `whole` / `prefix` / `supplement` / `number` / `body`) layers per part. See "Fine-grained caption text" section.
`caption-leading`	`0.5em`	Caption line spacing
`pre-supplement-number-spacing`	`auto`	Prefix–number spacing
`post-supplement-number-spacing`	`auto`	Number–suffix spacing
`number-title-spacing`	`auto`	Number–title separator
`number-title-spacing-en`	`auto`	Number–English-title separator
`lang`	`auto`	Language code override
`enable-english-caption`	`true`	Enable English sub-caption
`outline-bilingual`	`false`	Bilingual outline
`outline-separator`	`" / "`	Outline separator
`outline-newline`	`false`	Newline in outline
`after-indent`	`auto`	Post-block indent fix
`note-above`	`0.7em`	Above note

Param	Default	Description
`note-below`	`1em + 1.5pt`	Below note
`note-size`	`10.5pt`	Note size
`note-leading`	`6.5pt`	Note leading
`note-justify`	`true`	Justify note

Figure spacing section

Param	Default	Description
`figure-above`	`1em`	Space above figure
`figure-below`	`1em`	Space below figure
`caption-above`	`0.5em`	Image–caption gap
`caption-leading`	`0.5em`	Caption leading
`subcaption-above`	`0.3em`	Subfig–subcaption gap
`subcaption-below`	`0.5em`	Below subcaption
`subcaption-number-title-spacing`	`auto`	Separator between subcaption number and body (`auto` inherits main caption's `number-title-spacing`; accepts content / length)

Subfigure defaults section

Param	Default	Description
`gutter`	`1em`	Horizontal gap
`subcaption-pos`	`"bottom"`	Subcaption position
`show-subcaption`	`false`	Show subcaptions
`show-subcaption-label`	`true`	Subcaption includes label
`align`	`"horizon"`	Vertical alignment
`label-mode`	`none`	Label mode (`none` / `"overlay"`)
`label-style`	`"(a)"` / dict	Label style: pass `str` to share between overlay and subcaption; pass `(overlay: ..., subcaption: ...)` dict for per-side control (missing keys fall back to `"(a)"`)
`label-font`	`("Arial",)`	Label font list
`label-size`	`12pt`	Label size
`label-offset`	`(4pt, 4pt)`	Offset
`label-text-color`	`black`	Text color
`label-stroke`	`none`	Stroke
`label-bg`	`none`	Background
`label-bg-shape`	`"rect"`	Background shape
`label-bg-radius`	`2pt`	Rect radius

Param	Default	Description
`label-bg-inset`	`3pt`	Background padding
`label-sep`	`auto`	Subref separator (`auto`: number -> `"."`, letter -> `""`; defaults to `""` in `subref-style: "full"` mode)
`subref-style`	`"letter"`	Subref letter style: `"letter"` (just the letter, e.g. `Fig. 1a`) / `"full"` (with label-style decorations, e.g. `Fig. 1(a)`)
`caption-position`	`bottom`	Caption position relative to body in `#bicap()[body]` mode (figures default to `bottom`)
`caption-align`	`"center"`	Caption horizontal alignment: `"center"` / `"left"` / `"right"` / `"text-left"` / `"text-right"`; or dict `(main, continued)`
`placement`	`none`	Floating placement: `none` / `top` / `bottom` / `auto`

VIII.6 `captab` Complete Parameters

Param	Type	Description
`columns`	`auto` / `int` / `array`	Column config (preferred; `auto` or length array, supports `fr`/abs units)
`cols`	`auto` / `int` / `array`	Back-compat alias for `columns` (`columns` wins if both set)
`align`	`auto` / `alignment` / `array` / `function`	Cell alignment; merged with markdown `:---:` syntax (forwarded to tablem then to `table.align`)
`size`	`auto` / `length`	Body font size
`leading`	`auto` / `length`	Body line spacing
`inset`	`auto` / `length` / `dictionary`	Cell padding (alias of `cell-inset`; `cell-inset` wins if both passed)
`cell-inset`	`auto` / `length` / `dictionary`	Alias of `inset` (matches the global `captab-style.cell-inset` name)
`caption`	`none` / `content`	Main language caption
`caption-en`	`none` / `content`	English caption
`refer-to`	`none` / `label`	Label ref for continuation
`show-caption`	`auto` / `bool`	Show caption in continuation
`width`	`auto` / `length` / `ratio`	Table width (`auto` reads global, default `auto` unconstrained; pass `8cm` / `80%` / `80.5%`)
`caption-position`	`auto` / `top` / `bottom`	Caption position (`auto` reads global `caption-position`, default `top`; `bottom` places the caption

Param	Type	Description
		below the table; combined with `continued-caption: true` the latter is silently disabled)
`caption-align`	`auto` / `str` / `dict`	Caption horizontal alignment: `"center"` / `"left"` / `"right"` (table-local) / `"text-left"` / `"text-right"` (text-area-local); or dict `(main, continued)` to split; `auto` reads global
`placement`	`none` / `top` / `bottom` / `auto`	Floating placement (`none` in-flow, `top`/`bottom` to page edge, `auto` lets Typst pick the closer of top/bottom); omit to follow global state (initially `none`); forces `breakable: false` when active
`three-line-table`	`auto` / `bool`	Three-line mode (`auto` reads global `three-line-table`, default `true`)
`top-rule`	`auto` / `stroke`	Top rule stroke (`auto` reads global `top-rule`, default `1.5pt`; only used when `three-line-table: true`)
`middle-rule`	`auto` / `stroke`	Middle rule stroke (`auto` reads global `middle-rule`, default `0.5pt`)
`bottom-rule`	`auto` / `stroke`	Bottom rule stroke (`auto` reads global `bottom-rule`, default `1.5pt`)
`extra-rule`	`auto` / `stroke` / `dict`	Default stroke for `hlines` / `vlines` (when entries omit `stroke`); single value or `(h: ..., v: ...)` dict for per-axis (missing keys fall back to `0.5pt`); per-line `stroke` still wins
`breakable`	`auto` / `bool`	Allow page break (`auto` reads global `breakable`, default `true`)
`repeat-header`	`auto` / `bool` / `int`	Repeat header row on continuation pages (`auto` reads global `repeat-header`)
`continued-caption`	`auto` / `bool`	Repeat caption on each continuation page (refer-to format)
`hlines`	`array` of `dictionary`	Extra horizontal rules (see below)
`vlines`	`array` of `dictionary`	Extra vertical rules (see below)
`label`	`none` / `label`	Cross-reference label
`content`	`content` (positional)	Markdown-style body

`hlines` / `vlines` dict structure:

Key	hlines default	vlines default	Description
`row` / `col`	2	1	Row above (h) or col right (v)

Key	hlines default	vlines default	Description
start	0	0	Start col/row index
end	none	none	End index (none = to end)
stroke	0.5pt	0.5pt	Rule stroke

```
#captab(
  hlines: (
    (row: 2, stroke: 1pt),
    (row: 4, start: 1, end: 3, stroke: 0.3pt + red),
  ),
  vlines: (
    (col: 1, start: 1, stroke: 0.5pt),
  ),
  caption: [Complex Rules],
)[...]
```

Table 19: Complex Rules

A	B	C	D
1	2	3	4
5	6	7	8
9	0	1	2

VIII.6.1 Continuation Mode Examples

Prefix mode (default): shows `"Continuation of Table 1.1"`

Suffix mode: shows `"Table 1.1 (continued)"`

```
#show: captab-style.with(continued-mode: "suffix")
#captab(caption: [Long Table], label: <tab:long2>)[...]
#captab(refer-to: <tab:long2>)[...]  // Shows "Table 1.1 (continued)"
```

Force show / hide caption text:

```
// Force show
#captab(
  refer-to: <tab:original>,
  caption: [Original Caption],
  show-caption: true,
)[...]

// Force hide (even when caption is provided)
```

```
#captab(
  refer-to: <tab:original>,
  show-caption: false,
)[...]
```

VIII.7 bicap Standalone Caption Function

bicap is the standalone caption generator; usable independently of captab/capfig for custom layouts or embedding bilingual captions in plain Typst tables/figures.

Starting from 0.1.0 bicap supports two calling forms:

- Caption only: #bicap(caption: ..., kind: ...) — renders the caption alone in its own non-breakable block.
- Caption + body: #bicap(caption: ..., kind: ...)[body] — wraps body and the caption together in a single non-breakable block, conceptually a cap-able-flavoured figure(body, caption: ...). The body may also be passed via the named argument body: [...].

The relative position of caption vs. body is controlled by caption-position:

- The default follows the global state — top for tables, bottom for figures.
- Override globally via captab-style / capfig-style with caption-position: top | bottom.
- Override per-call by passing position: top | bottom to bicap.

VIII.7.1 Caption horizontal alignment caption-align

caption-align controls how the caption sits horizontally relative to the table/figure and the text area. Five values:

Value	Meaning
"center" (default)	Caption centered within the table/figure's own width (current behaviour)
"left"	Aligned to the table/figure's left edge (table-local)
"right"	Aligned to the table/figure's right edge (table-local)
"text-left"	Aligned to the text-area left edge (regardless of table/figure width)
"text-right"	Aligned to the text-area right edge

When width: 100% the table/figure fills the text width, so "left" ≡ "text-left" and "right" ≡ "text-right" collapse naturally.

Splitting main vs continuation captions — pass a dict:

```
#captab(
  caption-align: (main: "left", continued: "right"),
  continued-caption: true,
```

```
4   ...
5 )[ ... ]
```

`main` controls the first-page caption; `continued` controls the "Cont. Table X.Y" caption that appears on each break page when `continued-caption: true`. Missing keys fall back to `"center"`.

Exposed at:

- Global: `cap-style(caption-align: ...)`, `captab-style`, `capfig-style`
- Per-call: `captab(caption-align: ...)`, `bicap(caption-align: ...)`

Known limitation: continuation `text-left` / `text-right` silently degrade

The continuation caption emitted by `continued-caption: true` is implemented as a cap-cell inside `table.header(level: 1, repeat: true)` — structurally locked to the table's column width, with no way to extend out to the text-area edges. We considered escape hatches via `place()` and `set page(footer: ...)` but each introduced new bugs (unstable coordinates, clobbering user-defined footers), so we picked an honest degradation over unstable magic.

When `caption-align` (or the `continued` side of a dict) is `"text-left"` / `"text-right"`, the continuation side silently falls back to `"left"` / `"right"` (aligned to the table's column width). The main caption side is unaffected.

Workaround when needed — manual `refer-to`: split the table into segments and write each continuation segment with a separate `captab(refer-to: <main>, caption-align: "text-left", ...)`. The continuation caption then lives outside the original table at full text width, and all five alignment values work precisely.

VIII.7.2 Floating placement `placement`

Mirrors Typst's native `figure(placement: ...)`: lifts the table/figure out of the document flow and floats it to the top or bottom of the current page (or the next one), letting body text fill the remaining space. Equivalent to LaTeX's `[t]` / `[b]`.

Value	Meaning
`none` (default)	Render in-flow (no float)
`top`	Float to the top of the current or next page
`bottom`	Float to the bottom of the current or next page
`auto`	Let Typst pick `top` or `bottom` automatically — whichever edge is closer to the current document position

```
1 #captab(caption: ..., placement: top)[ ... ]      // float top
2 #captab(caption: ..., placement: bottom)[ ... ]   // float bottom
3 #capfig(caption: ..., placement: top)[ ... ]
```

```
#bicap(placement: top, ...)[ body ]

// Global: every table floats to the top
#show: captab-style.with(placement: top)
```

Exposed at:

- Global: `cap-style(placement: ...)`, `captab-style`, `capfig-style`
- Per-call: `captab` / `capfig` / `capsubfig` / `bicap`

Known limitations:

1. Floats can't break across pages. Typst's `place(float: true)` requires the floated content to fit on a single page, so:
   - When `placement: top` / `bottom` is active, cap-able forces `breakable: false` on the inner block — even if the user passes `breakable: true`.
   - A long table (taller than one page) will overflow / clip when floated. Keep `placement: none` for such tables and let them flow naturally.
2. Incompatible with `continued-caption: true`. Continuation depends on page breaks, which floats forbid; the combination effectively disables `continued-caption` (no continuation pages exist).
3. Cross-references `@ref`: cap-able wraps the hidden figure (counter-registration anchor) inside the float wrapper too, so `@ref` resolves to the float's landing page — matching what the reader sees. This mirrors Typst's native `figure(placement: ...)` behaviour.

Param	Type	Description
`caption`	`none` / `content`	Main caption
`caption-en`	`none` / `content`	English caption
`refer-to`	`none` / `label`	Continuation ref label
`label`	`none` / `label`	Label for cross-ref
`kind`	`"table"` / `"figure"`	Kind (selects counter and config source)
`show-caption`	`auto` / `bool`	Show caption in continuation
`config`	`auto` / `dictionary`	`auto` reads the global config matching `kind`; otherwise uses the dict
`position`	`auto` / `top` / `bottom`	Caption position relative to body (only when body is given; `auto` follows global `caption-position`)
`caption-align`	`auto` / `str` / `dict`	Caption horizontal alignment: `"center"` / `"left"` / `"right"` / `"text-left"` / `"text-right"`, or dict `(main, continued)`; `auto` reads global

Param	Type	Description
placement	none / top / bottom / auto	Floating placement (auto = Typst picks the closer of top/bottom); forces breakable: false when active
breakable	bool	Whether the outer block may break across pages (default true; lets a breakable body flow naturally)
continued-caption	bool	When body breaks across pages, repeat the caption above each native #table() in body (default false; only when kind == "table")
repeat-header	auto / bool / int	Override the repeat setting of the markdown header in body's #table() (auto leaves user's value alone; true/false/int overrides; only when kind == "table")
body	none / content	Optional body; may also be passed via trailing block #bicap()[...]

```
// Form 1: caption only (same as 0.0.x)
#align(center)[
  #rect(width: 6cm, height: 3cm, fill: gray.lighten(70%))
  #bicap(
    caption: [Custom Figure],
    caption-en: [Custom Figure],
    kind: "figure",
    label: <fig:custom>,
  )
]

// Form 2: bicap wraps the body (kind=table; caption defaults to top)
#bicap(
  caption: [Experimental Data],
  kind: "table",
  label: <tab:exp>,
)[
  #table(
    columns: 3,
    [A], [B], [C],
    [1], [2], [3],
  )
]

```

```
// kind=figure; caption defaults to bottom — use position: top to flip
#bicap(
  caption: [Schematic],
  kind: "figure",
  position: top,
)[
  #image("foo.png", width: 6cm)
]

// Long table that should repeat the caption on each continuation page (table-kind only)
#bicap(
  caption: [Long Data Table],
  kind: "table",
  continued-caption: true,
  label: <tab:long>,
)[
  #table(
    columns: 4,
    table.header([ID], [Name], [Value], [Note]),
    // ... 50+ rows of data
  )
]

// Repeat only the caption on continuation pages — turn off markdown header repetition
#bicap(
  caption: [Long Table],
  kind: "table",
  continued-caption: true,
  repeat-header: false,
)[
  #table(
    columns: 4,
    table.header([Col1], [Col2], [Col3], [Col4]),
    // ...
  )
]

// Or just disable markdown header repetition on its own, without continued-caption
#bicap(
  caption: [Short Table],
  kind: "table",
  repeat-header: false,
)[ #table(columns: 3, table.header([A], [B], [C]), ...) ]
```

Convention: keep at most one `#table()` per bicap. When `continued-caption: true`, bicap installs a `show table:` rule that injects the SAME caption header into every native `#table()` in body. If you stuff multiple tables under one bicap, every table's continuation page will show the same caption — usually not what you want. Use multiple `bicap(...)` calls or manage each table's caption with `captab`.

Note: bicap detects `captab`'s own nested level $\geqslant 2$ header structure and skips the injection in that case (so wrapping `captab(continued-caption: true)` inside bicap won't produce stacked captions). Still, avoid nesting captab inside a bicap with `continued-caption` — the semantics get muddled and numbering can drift.

VIII.8 `capfig` Complete Parameters

Param	Type	Description
`content`	`content` (positional)	Figure body
`caption`	`none` / `content`	Main language caption
`caption-en`	`none` / `content`	English caption
`label`	`none` / `label`	Cross-reference label
`refer-to`	`none` / `label`	Continuation ref label
`show-caption`	`auto` / `bool`	Show caption in continuation
`figure-above`	`auto` / `length`	Space above figure (`auto` = global)
`figure-below`	`auto` / `length`	Space below figure (`auto` = global)
`caption-above`	`auto` / `length`	Image–caption gap (`auto` = global)
`caption-leading`	`auto` / `length`	Caption leading (`auto` = global)

The figure body and caption are always wrapped in a non-breakable block; the caption is always placed below the image; if `capfig-style.lang` is `auto` while `captab-style.lang` is not, the figure inherits the table's language setting.

VIII.9 `capsubfig` Complete Parameters

VIII.9.1 Subfigure Dict Structure

`subfigs` is an array of dicts; each supports:

Key	Required	Description
`content`	Yes	Subfigure body
`subcaption`	Cond.	Subcaption (used when `show-subcaption: true`)
`label`	Opt.	Cross-ref label, e.g. `Figure 1.1a`
`label-style-override`	Opt.	Per-item style override (overlay), see below

VIII.9.2 `label-style-override` dict keys

Per-subfigure overrides for the overlay label. Affects the overlay side only; the subcaption does not participate.

Key	Overrides
`style`	`label-style.overlay` (or `label-style` string form) — per-subfig overlay format string, e.g. `"[I]"`, `"{1}"`
`font`	label font list
`size`	label font size
`offset`	label `(dx, dy)` offset
`text-color`	label text color
`stroke`	label stroke
`bg`	label background
`bg-shape`	background shape
`bg-radius`	rect radius
`bg-inset`	background padding

`style` also affects cross-references: when ref follows overlay (i.e. subcaption is hidden), `@ref` renders the per-subfig letter in that subfig's own format. When subcaption is visible, ref still follows the global subcaption style; the per-subfig `style` only changes the figure-stamped label.

VIII.9.3 Function Parameters

All params besides `subfigs` accept `auto` (inherit global `capfig-style`):

- `columns` (`auto` / `int`): Columns per row; `auto` = single row.
- `caption` / `caption-en` / `label` / `refer-to` / `show-caption`: Same as `capfig`.
- `gutter`, `subcaption-pos`, `show-subcaption`, `show-subcaption-label`, `align`, `label-mode`, `label-style`, `label-font`, `label-size`, `label-offset`, `label-text-color`, `label-stroke`, `label-bg`, `label-bg-shape`, `label-bg-radius`, `label-bg-inset`, `label-sep`, `subref-style`: Override the subfigure defaults from `capfig-style`.
  - `label-style` accepts `str` (overlay/subcaption share the style) or `(overlay: ..., subcaption: ...)` dict for per-side control. Missing keys fall back to the package default `"(a)"`.
- `figure-above`, `figure-below`, `caption-above`, `subcaption-above`, `subcaption-below`: Spacing overrides.
- `subcaption-number-title-spacing` (`auto` / `content` / `length`): separator between subcaption number and body; `auto` inherits the main caption's `number-title-spacing`.

VIII.9.4 Label Style Parse Rules

The parser scans `label-style` left-to-right, locates the first format char (`a`/`A`/`1`/`i`/`I`); everything before is prefix, everything after is suffix. If no format char exists, the whole string becomes prefix with default format `"a"`.

- Roman numerals support 1–20; fallback to Arabic.
- `label-sep auto` rule: number format -> `"."` (e.g. `"Figure 1.1.2"`), letter/roman format -> `""` (e.g. `"Figure 1.1a"`).

VIII.9.5 Overlay with Circle Background

```
#capsubfig(
  (
    (content: rect(width: 4cm, height: 2.5cm, fill: yellow.lighten(70%))),
    (content: rect(width: 4cm, height: 2.5cm, fill: red.lighten(70%)),
     label-style-override: (bg: green, text-color: white)),
  ),
  columns: 2,
  label-mode: "overlay",
  label-style: "(1)",
  label-bg: blue.lighten(60%),
  label-bg-shape: "circle",
  label-size: 10pt,
  caption: [Circle bg + per-item override],
)
```

VIII.9.6 Multi-row Subfigures

Set `columns: N`; overflow auto-wraps, the last row shrinks to the actual count.

```
#capsubfig(
  (
    (content: rect(width: 2cm, height: 2cm, fill: red)),
    (content: rect(width: 2cm, height: 2cm, fill: green)),
    (content: rect(width: 2cm, height: 2cm, fill: blue)),
    (content: rect(width: 2cm, height: 2cm, fill: orange)),
    (content: rect(width: 2cm, height: 2cm, fill: purple)),
  ),
  columns: 3,                // 3 per row: 3+2
  label-mode: "overlay",
  caption: [Five Subfigures],
)
```

VIII.10 `captnote` Complete Parameters

Param	Type	Description
`width`	`auto` / `ratio` / `length`	Width (three modes below)
`justify`	`auto` / `bool`	Justify (`auto` = global `note-justify`)
`content`	`content` (positional)	Table note body

Three width modes:

- `width: auto` (default): Match current `captab` width via `table-width-config`.
- `width: 80%` etc. `ratio`: Narrow to percentage, centered.
- `width: 10cm` etc. `length`: Fixed width, centered.

```
#captnote[Auto match table width]
#captnote(width: 70%)[70% width centered]
#captnote(width: 8cm)[Fixed 8cm]
#captnote(justify: false)[No justification]
```

VIII.11 capfnote Complete Parameters

Param	Type	Description
`width`	`ratio` / `length`	Default `100%`; `auto` not supported
`justify`	`auto` / `bool`	Justify (`auto` = global `capfig-style.note-justify`)
`content`	`content`	Figure note body

```
#capfig(image("chart.png"), caption: [Results])
#capfnote(width: 90%)[
  Source: 2024 Survey.
]
```

VIII.12 Full Multilingual Rules

The following table lists prefix words and spacing rules for each language (`pre` = `pre-supplement-number-spacing`, `sep` = `number-title-spacing`):

lang	table	figure	pre	sep	continued-suffix
en	Table	Figure	" "	": "	(continued)
zh	表	图	0em	U+3000	（续）
zh-TW	表	圖	0em	U+3000	（續）
de	Tabelle	Abbildung	" "	": "	(Fortsetzung)
fr	Tableau	Figure	" "	" : "	(suite)
es	Tabla	Figura	" "	". "	(continuación)
it	Tabella	Figura	" "	". "	(continua)
pt	Tabela	Figura	" "	": "	(continuação)
ru	Таблица	Рисунок	" "	". "	(продолжение)
ja	表	図	0em	U+3000	（続き）
ko	표	그림	" "	". "	(계속)
ar ↲	جدول	شكل	" "	": "	(تابع)
nl	Tabel	Figuur	" "	": "	(vervolg)
pl	Tabela	Rysunek	" "	". "	(cd.)
cs	Tabulka	Obrázek	" "	": "	(pokračování)

lang	table	figure	pre	sep	continued-suffix
sv	Tabell	Figur	" "	". "	(forts.)
da	Tabel	Figur	" "	": "	(fortsat)
no	Tabell	Figur	" "	": "	(forts.)
fi	Taulukko	Kuva	" "	". "	(jatkoa)
tr	Tablo	Şekil	" "	". "	(devam)
el	Πίνακας	Σχήμα	" "	". "	(συνέχεια)
he ↲	טבלה	תמונה	" "	": "	(המשך)
hi	तालिका	चित्र	" "	": "	(जारी)
th	ตาราง	รูป	" "	" "	(ต่อ)
vi	Bảng	Hình	" "	". "	(tiếp theo)
fa ↲	جدول	شکل	" "	": "	(ادامه)
ur ↲	جدول	شکل	" "	": "	(جاری)

↲ = Right-to-left language

Auto-triggers post-table indent fix (`after-indent: auto`) for: `zh`, `ja`, `ko`, `fr`, `vi`, `th`. Any language can override via explicit `after-indent: true/false`.

RTL bilingual layout: for `ar`/`he`/`fa`/`ur`, the main-language and English lines are auto-swapped, the English line is wrapped in `#text(dir: ltr)[...]` to prevent direction confusion, and in monolingual mode the main-language body is wrapped in `box[]`.

Part IX

API Reference

This chapter provides auto-generated documentation from source code docstrings via tidy.

IX.1 Configuration Module (config.typ)

The configuration module provides global state management, multilingual text, utility functions, and the two main configuration functions.

- cap-style()
- capfig-style()
- captab-style()
- resolve-lang-indent()
- set-figure-width()
- set-table-width()

IX.1.1 cap-style

同时为表格和图片设置全局样式。共享参数会同时作用于两者；之后再调用 captab-style 或 capfig-style 可对某一类进行覆盖。Configure both table and figure global styles at once. Shared params are applied to both; a subsequent captab-style / capfig-style call can override for one type only.

Parameters

```
cap-style(
  numbering-format,
  use-chapter,
  supplement,
  supplement-en,
  continued-prefix,
  continued-prefix-en,
  continued-suffix,
  continued-suffix-en,
  continued-mode,
  continued-show-caption,
  caption-size,
  caption-weight,
  caption-text,
  caption-leading,
  caption-above,
  pre-supplement-number-spacing,
  post-supplement-number-spacing,
  number-title-spacing,
  number-title-spacing-en,
  lang,
  enable-english-caption,
  outline-bilingual,
  outline-separator,
  outline-newline,
  after-indent,
  note-above,
  note-below,
  note-size,
  note-leading,
  note-justify,
  ,
  caption-position,
  caption-align,
  placement,
  it
) -> content
```

IX.1.2 capfig-style

Configure global figure and subfigure style within the body. 所有参数默认 auto = “保持当前 state 不变”，多次调用做 patch（增量覆盖）。 All params default to auto = “keep current state”. Multiple calls patch incrementally.

Parameters

```
capfig-style(
  numbering-format,
  use-chapter,
  supplement,
  supplement-en,
  continued-prefix,
  continued-prefix-en,
  continued-suffix,
  continued-suffix-en,
  continued-mode,
  continued-show-caption,
  caption-size,
  caption-weight,
  caption-text,
  pre-supplement-number-spacing,
  post-supplement-number-spacing,
  number-title-spacing,
  number-title-spacing-en,
  lang,
  enable-english-caption,
  outline-bilingual,
  outline-separator,
  outline-newline,
  after-indent,
  figure-above,
  figure-below,
  caption-above,
  caption-leading,
  subcaption-above,
  subcaption-below,
  subcaption-number-title-spacing,
  gutter,
  subcaption-pos,
  show-subcaption,
  show-subcaption-label,
  align,
  label-mode,
  label-style,
  label-font,
  label-size,
  label-offset,
  label-text-color,
  label-stroke,
  label-bg,
  label-bg-shape,
  label-bg-radius,
  label-bg-inset,
  label-sep,
  subref-style,
  note-above,
  note-below,
  note-size,
  note-leading,
  note-justify,
  caption-position,
  caption-align,
  placement,
  it
```

IX.1.3 captab-style

Configure global table style for all cap-table, bicap and table-note calls within the body. 所有参数默认为 auto = “保持当前 state 不变”，因此可以多次调用 captab-style 仅覆盖部分字段（patch 语义），而不会把未提供的字段重置为默认值。All params default to auto = “keep current state”. Multiple captab-style calls patch only the supplied fields; omitted fields are preserved.

Parameters

```
  captab-style(
    caption-above,
    caption-below,
    table-below,
    caption-leading,
    numbering-format,
    use-chapter,
    supplement,
    supplement-en,
    continued-prefix,
    continued-prefix-en,
    continued-suffix,
    continued-suffix-en,
    continued-mode,
    continued-show-caption,
    caption-size,
    caption-weight,
    caption-text,
    pre-supplement-number-spacing,
    post-supplement-number-spacing,
    number-title-spacing,
    number-title-spacing-en,
    lang,
    enable-english-caption,
    body-size,
    body-leading,
    cell-inset,
    inset,
    note-above,
    note-below,
    note-size,
    note-leading,
    note-justify,
    outline-bilingual,
    outline-separator,
    outline-newline,
    after-indent,
    width,
    three-line-table,
    top-rule,
    middle-rule,
    bottom-rule,
    extra-rule,
    breakable,
    repeat-header,
    continued-caption,
    caption-position,
    caption-align,
    placement,
    it
  ) -> content
```

IX.1.4 resolve-lang-indent

共享语言与缩进修复解析函数。给定含 lang/after-indent 字段的配置， 返回 (lang, should-indent)。供 captab/capfig/captnote/capfnote 复用。Shared language/indent resolution. Given a config dict with lang/after-indent fields, returns (lang, should-indent). Used by captab/capfig/captnote/capfnote.

Parameters

```
resolve-lang-indent(config)
```

IX.1.5 set-figure-width

Set figure width percentage (1-100) for capfnote auto-width matching.

Parameters

```
set-figure-width(percentage) -> content
```

IX.1.6 set-table-width

Set table width globally. Accepts `auto` (no fixed width), a `length` (e.g. `8cm`) or a `ratio` (e.g. `80%`, `80.5%`).

兼容旧 API：`set-table-width(percentage: 80)` 仍然可用，会被转成 `80%`。 Backward-compat: legacy `set-table-width(percentage: <int>)` is still accepted and converted to `<int>%` internally.

Parameters

```
set-table-width(
  width,
  ,
  percentage
) -> content
```

IX.2 Caption Module (bicap.typ)

The caption module provides the core bilingual caption generation engine, supporting both main and continuation (continued table/figure) modes.

- bicap()

IX.2.1 bicap

独立双语标题函数，可用于表格（`kind: "table"`）或图片（`kind: "figure"`）。

调用方式 / Usage:

- `#bicap(caption: [...], kind: "...")` —— 仅渲染题注，按 `caption-above` / `caption-below` 间距独立成块。
- `#bicap(caption: [...], kind: "...")[body]` —— 把 body 与题注一起包进一个 不换页 block，相当于一个 `figure(kind: ...)` 的 cap-able 版本。题注与 body 的相对位置由 `caption-position` 决定（state 默认：table=top、figure=bottom；可通过 `captab-style` / `capfig-style` 全局覆盖，或直接传 `config`）。

Standalone bilingual caption for tables (`kind: "table"`) or figures (`kind: "figure"`).

Calling forms:

- `#bicap(caption: [...], kind: "...")` — caption only, in its own non-breakable block.
- `#bicap(caption: [...], kind: "...")[body]` — wraps body and caption together, conceptually a cap-able-flavoured `figure(kind: ...)`. Caption position is taken from `caption-position` (default top for tables, bottom for figures; configurable via `captab-style` / `capfig-style` or by passing `config`).

Parameters

```
bicap(
  caption: none content,
  caption-en: none content,
  refer-to: none label,
  label: none label,
  kind: str,
  show-caption: auto bool,
  config: auto dictionary,
  position: auto alignment,
  caption-align: auto str dictionary,
  placement: auto none alignment,
  breakable: bool,
  continued-caption: bool,
  repeat-header: auto bool int,
  body: none content,
  ..body-args
) -> content
```

caption none or content

主语言标题文本 Main language caption text

Default: none

caption-en none or content

英文标题文本（双语模式） English caption text (bilingual mode)

Default: none

refer-to `none` or `label`

引用原始表/图的标签（提供此参数则进入续表模式）Reference to the original table/figure label (enables continuation mode)

Default: `none`

label `none` or `label`

此标题的标签（用于交叉引用） Label for this caption (for cross-references)

Default: `none`

kind `str`

内容类型：“table” 或 “figure” Content type: “table” or “figure”

Default: `"table"`

show-caption `auto` or `bool`

续表/图是否显示标题文本（auto=自动根据 caption 是否提供决定） Whether to show caption text in continuation (auto = based on whether caption is provided)

Default: `auto`

config `auto` or `dictionary`

样式配置，auto 则从全局 table-style-config 获取 Style config, auto = use global table-style-config

Default: `auto`

position `auto` or `alignment`

题注位置（auto = 跟随全局 caption-position；top 或 bottom 直接覆盖）Caption position (auto = follow global caption-position; top / bottom to override)

Default: `auto`

caption-align `auto` or `str` or `dictionary`

题注水平对齐：字符串 “center” / “left” / “right” / “text-left” / “text-right” 也接受 dict (main: ..., continued: ...) 拆分主与续。auto = 跟随全局 caption-align。Caption horizontal alignment: string “center” / “left” / “right” / “text-left” / “text-right”. Also accepts dict (main, continued) for split. auto = follow global.

Default: `auto`

placement `auto` or `none` or `alignment`

浮动定位：none（默认，原地）/ top / bottom（浮到当前页或下一页的顶/底）。auto = 跟随全局 placement。启用时强制 breakable: false（Typst float 不允许跨页）。 Floating placement; auto follows global. Forces breakable: false when active.

Default: `()`

breakable `bool`

外层 block 是否允许跨页。默认 `true`, 让 body 里本身可跨页的内容（如长 captab、长段落）顺延到下一页；设为 `false` 则强制把题注 + body 锁在同一页。 Whether the outer block may break across pages. Defaults to `true` so a body that is itself breakable (e.g. a long captab or paragraph) flows naturally; set `false` to keep caption + body atomic on one page.

Default: `true`

continued-caption `bool`

跨页时是否在 body 里 **每张原生**

的顶部重复题注。 仅在 `kind == "table"` 时生效；通过 `show table:` 注入一个 level-1 `table.header(repeat: true)` 实现，复用与 `captab(continued-caption: true)` 同款的 `query(label) + here().page()` 检测，续页才显示。

约定：一个 bicap 里只放一张 `#table()`。如果 body 里同时有多张表，所有表都会被 注入相同的题注（因为 bicap 把它们视作“同一题注下的多个表”），通常不是你想要的； 此时请改写成多个 bicap 调用，或用 `captab` 直接管理每张表的题注。

`kind == "figure"` 下此参数为 no-op（图片本身不跨页；如有跨页图请用 `refer-to`）。

Whether to repeat the caption above each native `#table()` inside body when it breaks across pages. Only active when `kind == "table"`. Implemented by injecting a level-1 `table.header(repeat: true)` via `show table:`, reusing the `query(label) + here().page()` mechanism from `captab(continued-caption: true)` so the repeated caption only renders on continuation pages.

Convention: keep at most one `#table()` per bicap. If body contains multiple tables, the same caption is injected into ALL of them — usually not what you want; in that case use multiple bicap calls or manage each table's caption with `captab`.

No-op when `kind == "figure"`; for cross-page figures use `refer-to`.

Default: `false`

repeat-header `auto` or `bool` or `int`

跨页时是否重复 body 内 `#table()` 的 markdown 表头行。 `auto`（默认）= 不干涉用户写在 `table.header(...)` 里的 `repeat` 设定； `true | false | <int>` = 强制覆盖，跟 Typst 原生 `table.header(repeat: ...)` 一致。 仅 `kind == "table"` 生效， 与 `continued-caption` 互相独立。

Whether to repeat the markdown header row of `#table()` inside body across pages. `auto` (default) leaves the user's existing `table.header(repeat: ...)` untouched; `true | false | <int>` overrides it (matching Typst's native semantics). Only active when `kind == "table"`; independent of `continued-caption`.

Default: `auto`

body `none` or `content`

可选的 body：可以用名参 `body: [...]` 或在调用尾部用内容块 `#bicap()[...]`，后者会通过 `..body-args` 收集进来。两种形式等价。Optional body — pass via the named arg `body: [...]` or as a trailing content block `#bicap()[...]` (collected through `..body-args`); the two forms are equivalent.

Default: `none`

IX.3 Table Module (table.typ)

The table module provides academic-standard three-line table creation and its aliases.

- captab()

IX.3.1 captab

创建三线表，支持 Markdown 语法、双语标题、续表和自定义线条。Create a three-line table with Markdown syntax, bilingual captions, continuation, and custom lines.

- columns (auto | int | array): 列配置 / Column config（推荐用法 / preferred）
- cols (auto | int | array): columns 的向后兼容别名 / Back-compat alias for columns
- align (auto | alignment | array | function): 单元格对齐（与 tablem 的 :—: 语法合并）/ Cell alignment
- size (auto | length): 表格内容字号 / Table content font size
- leading (auto | length): 表格内容行距 / Table content line spacing
- inset (auto | length | dictionary): 单元格内边距 / Cell padding
- caption (none | content): 主语言标题 / Main language caption
- caption-en (none | content): 英文标题 / English caption
- refer-to (none | label): 续表引用的原表标签 / Original label for continuation
- show-caption (auto | bool): 续表是否显示标题文本 / Show caption in continuation
- three-line-table (auto | bool): 是否使用三线表样式（默认 true；false 时跳过手动三线，使用 Typst 默认 table 边框）/ Three-line table mode
- top-rule (auto | stroke): 顶线粗细，接受任何 stroke 值（如 1.5pt + red）/ Top rule stroke
- middle-rule (auto | stroke): 中线粗细 / Middle rule stroke
- bottom-rule (auto | stroke): 底线粗细 / Bottom rule stroke
- breakable (auto | bool): 表格能否跨页（默认从全局读取，初始 true）/ Whether table may break across pages
- repeat-header (auto | bool | int): 跨页时重复表头行 / Repeat the markdown header row on page-break
- continued-caption (auto | bool): 跨页时是否在每页表头之上重复题注（题注会进入 `table.header` 内）/ Repeat caption on every page

- hlines (array): 额外横线，每项为 `(row: int, start: int, end: none|int, stroke: stroke)` / Extra h-rules
- vlines (array): 额外竖线，每项为 `(col: int, start: int, end: none|int, stroke: stroke)` / Extra v-rules
- label (none | label): 表格标签，用于交叉引用 / Label for cross-reference
- ..extra-args: 任何其它命名参数（如 `fill`, `stroke`, `gutter`, `column-gutter`, `row-gutter`, `rows`）会原样透传给底层 `table(...)`, 与 `tablem` 的高级用法保持一致。`stroke` 用户传值时会覆盖我们三线模式默认的 `stroke: none`（建议这种情况下同时设 `three-line-table: false`，否则三条手画 hline 会叠在用户 stroke 之上）。Any other named args (e.g. `fill`, `stroke`, `gutter`, `column-gutter`, `row-gutter`, `rows`) are passed through to the underlying `table(...)`, mirroring tablem's advanced-usage surface. A user-provided `stroke` overrides the `stroke: none` we use in three-line mode (consider also passing `three-line-table: false`, otherwise the three manual hlines stack on top of user stroke).
- content (content): Markdown 风格的表格内容 / Markdown-style table content

Parameters

```
captab(
  columns,
  cols,
  align,
  size,
  leading,
  inset,
  cell-inset,
  caption,
  caption-en,
  refer-to,
  show-caption,
  caption-position,
  caption-align,
  placement,
  width,
  three-line-table,
  top-rule,
  middle-rule,
  bottom-rule,
  extra-rule,
  breakable,
  repeat-header,
  continued-caption,
  hlines,
  vlines,
  label,
  ..extra-args,
  content
)
```

IX.4 Note Module (note.typ)

The note module provides table and figure notes with auto-width matching and multiple width modes.

- capfnote()
- captnote()

IX.4.1 capfnote

创建图片注释（图注），从 `figure-style-config` 读取样式，默认宽度自动匹配。Create a figure note; reads style from `figure-style-config`, auto-matches figure width.

- width (auto | ratio | length): 图注宽度 / Note width
- justify (auto | bool): 是否两端对齐 / Text justification
- content (content): 图注内容 / Note content

Parameters

```
capfnote(
  width,
  justify,
  content
)
```

IX.4.2 captnote

创建表格注释（表注），默认宽度自动匹配 *captab* 表格宽度。Create a table note; default width auto-matches *captab* width.

- width (auto | ratio | length): 表注宽度 / Note width
- justify (auto | bool): 是否两端对齐（auto=使用全局配置）/ Text justification
- content (content): 表注内容 / Note content

Parameters

```
captnote(
  width,
  justify,
  content
)
```

IX.5 Figure Module (figure.typ)

The figure module provides single figure and multi-subfigure layout functions, supporting bilingual captions, overlay labels, and subcaptions.

- capfig()
- capsubfig()

IX.5.1 capfig

创建带双语标题的单个图片，标题始终在图片下方。 Create a single figure with bilingual caption; caption always below image.

- content (content): 图片内容（通常为 `image()` 调用）/ Figure content (usually `image()`)
- caption (none | content): 主语言标题 / Main language caption
- caption-en (none | content): 英文标题 / English caption
- label (none | label): 图片标签，用于交叉引用 / Figure label for cross-references
- refer-to (none | label): 续图引用的原图标签 / Original label for continued figure
- show-caption (auto | bool): 续图是否显示标题文本 / Show caption in continued figure
- figure-above (auto | length): 图片上方间距 / Space above figure
- figure-below (auto | length): 图片下方间距 / Space below figure
- caption-above (auto | length): 标题与图片间距 / Space between image and caption
- caption-leading (auto | length): 标题行距 / Caption line spacing

Parameters

```
capfig(
  content,
  caption,
  caption-en,
  label,
  refer-to,
  show-caption,
  figure-above,
  figure-below,
  caption-above,
  caption-leading,
  placement
)
```

IX.5.2 capsubfig

创建多子图网格布局，支持子标题、覆盖标签和主双语标题。 Create a multi-subfigure grid layout with subcaptions, overlay labels, and bilingual main caption.

每个子图为字典：`(content: ..., subcaption: ..., label: ..., label-style-override: (...))`。 Each subfigure is a dict: `(content: ..., subcaption: ..., label: ..., label-style-override: (...))`.

- subfigs (array): 子图字典数组 / Array of subfigure dicts
- columns (auto | int): 每行列数，auto=所有子图在一行 / Columns per row
- caption (none | content): 主标题（主语言）/ Main caption (main language)
- caption-en (none | content): 主标题（英文）/ Main caption (English)

- label (none | label): 主图标签 / Main figure label
- refer-to (none | label): 续图引用的原图标签 / Original label for continuation
- show-caption (auto | bool): 续图是否显示标题 / Show caption in continuation
- gutter (auto | length): 子图水平间距 / Horizontal gap between subfigures
- subcaption-pos (auto | str): 子标题位置 “top”/“bottom”
- show-subcaption (auto | bool): 是否显示子标题 / Show subcaptions
- show-subcaption-label (auto | bool): 子标题是否含标签 / Subcaption includes label
- align (auto | str): 垂直对齐 “top”/“horizon”/“bottom”
- label-mode (auto | none | str): 标签模式 / Label mode (none or “overlay”)
- label-style (auto | str): 标签样式，如 “(a)”/“图 a” / Label style
- label-font (auto | array): 标签字体 / Label font
- label-size (auto | length): 标签大小 / Label size
- label-offset (auto | tuple): 标签偏移 (dx, dy) / Label offset
- label-text-color (auto | color): 标签颜色 / Label text color
- label-stroke (auto | none | stroke): 标签描边 / Label stroke
- label-bg (auto | none | color): 标签背景 / Label background
- label-bg-shape (auto | str): 背景形状 / Background shape
- label-bg-radius (auto | length): 背景圆角 / Background radius
- label-bg-inset (auto | length): 背景内边距 / Background padding
- label-sep (auto | str): 子图引用分隔符 / Subfigure reference separator
- figure-above (auto | length): 图片上方间距 / Space above figure
- figure-below (auto | length): 图片下方间距 / Space below figure
- caption-above (auto | length): 主标题上方间距 / Space above main caption
- subcaption-above (auto | length): 子标题上方间距 / Space above subcaption
- subcaption-below (auto | length): 子标题下方间距 / Space below subcaption

Parameters

```
capsubfig(
  subfigs,
  columns,
  caption,
  caption-en,
  label,
  refer-to,
  show-caption,
  gutter,
  subcaption-pos,
  show-subcaption,
  show-subcaption-label,
  align,
  label-mode,
  label-style,
  label-font,
  label-size,
  label-offset,
  label-text-color,
  label-stroke,
  label-bg,
  label-bg-shape,
  label-bg-radius,
  label-bg-inset,
  label-sep,
  subref-style,
  figure-above,
  figure-below,
  caption-above,
  subcaption-above,
  subcaption-below,
  subcaption-number-title-spacing,
  placement
)
```

Part X

Frequently Asked Questions

X.1 Why Markdown syntax for tables?

The Markdown table syntax (via `tablem`) is:

- Familiar to most users
- Easy to read and edit
- Quick to type
- Compatible with many editors

X.2 How do I create tables without captions?

Simply omit the `caption` parameter:

```
#captab()[
  | A | B |
  | - | - |
  | 1 | 2 |
]
```

X.3 Can I use regular Typst table syntax?

Yes! Use `bicap` to add a caption to any native Typst table:

```
#align(center)[
  #bicap(
    caption: [Regular Typst Table],
    caption-en: [Regular Typst Table],
    kind: "table",
  )
  #table(
  columns: 3,
  [A], [B], [C],
  [1], [2], [3],
  )
]
```

Table 20: Regular Typst Table

A	B	C
1	2	3

X.4 How do I reference tables and figures?

Use Typst's standard label and reference system:

```
#captab(caption: [...], label: <tab:example>)[...]

See @tab:example for details.
```

X.5 Why doesn't my continued table show the caption?

This applies specifically to manual `refer-to` mode: by default, when `caption` is omitted (and `show-caption` is `auto`), the continuation only shows the "Cont. Table X.Y" number. To force the caption text:

```
#captab(
  refer-to: <tab:original>,
  caption: [Original Caption],  // provide caption text
  show-caption: true,           // or force it
)[...]
```

If you want the continuation caption to appear automatically on every continuation page when the table breaks, use the new `continued-caption` mode (since 0.1.0):

```
#captab(
  caption: [Original Caption],
  continued-caption: true,    // emits "Cont. Table X.Y Caption" on each continuation page
)[...]
```

See Tables in Detail → Continued Tables for the full comparison.

X.6 How do I fix missing indentation after tables in Chinese?

The package automatically detects the language and fixes indentation (for Chinese, Japanese, etc.).

For manual control:

```
#show: captab-style.with(
  after-indent: true,   // Force fix
  // or
  after-indent: false,  // Disable fix
)
```

X.7 How do I show bilingual captions in the table of contents?

```
#show: captab-style.with(
  outline-bilingual: true,      // Enable bilingual outline
  outline-separator: " / ",     // Separator
  outline-newline: false,       // true=newline, false=same line
)
```

X.8 Why no `capsubtab` (sub-tables)?

cap-able currently does not ship a `capsubtab`. Reasons:

1. Sub-tables are rare in academic typesetting. Most style guides do not have a dedicated section on sub-tables; the prevailing practice is to merge comparison data into a single larger table with an extra "Group" column rather than splitting into (a)(b) sub-tables. Sub-figures (`capsubfig`) are everywhere in the literature; sub-tables show up in only a small fraction of cases.
2. Page-break behaviour is harder. A side-by-side row of sub-tables is an atomic block — long content overflows rather than flowing across pages; sub-figures (typically images) are short enough that this rarely matters.
3. A workable manual recipe already exists. For two small tables side by side, `grid` + multiple `captab(caption: none)` wrapped in an outer `figure(kind: table, ...)` is enough; the only thing missing is automatic sub-table cross-references like `@tab:sub-a` → "1.2a", and most authors just write "as shown in Table 1.2(a)" inline without a label anyway.

> Status: an issue has been opened upstream to track this. No requests to date. If you actually need it and the manual `grid` recipe doesn't fit, please +1 the GitHub issue and we'll implement when demand materialises.

Example recipe for side-by-side small tables:

```
#figure(
  kind: table,
  supplement: [Table],
  caption: figure.caption(position: top)[Experimental vs. control group data],
  grid(
    columns: 2,
    column-gutter: 1em,
    align: top,
    [
      *(a) Experimental* \
```

```
      #captab(caption: none)[
        | ID | Value |
        | -- | ----- |
        | 1  | 100   |
        | 2  | 200   |
      ]
    ],
    [
      *(b) Control* \
      #captab(caption: none)[
        | ID | Value |
        | -- | ----- |
        | 1  | 95    |
        | 2  | 205   |
      ]
    ],
  ),
)<tab:cmp>
```

Table 21: Experimental vs. control group data

(a) Experimental

ID	Value
1	100
2	200

(b) Control

ID	Value
1	95
2	205

Reference: `@tab:cmp` resolves to “Table 21”. Sub-tables (a)(b) get no automatic labels; just write “as shown in Table 21(a)” inline.

Part XI
Changelog

XI.1 Version 0.1.0

New features:

- `captab` and `captab-style` gain a `breakable` parameter (bool, default `true`) — controls whether the three-line table can break across pages. The outer `block` now follows this flag, so long tables flow naturally instead of being clipped to one page.
- New `repeat-header` parameter (bool / positive int, default `true`) — repeats the markdown header row on each continuation page using Typst's native `table.header(repeat: ...)`. `true` = repeat on every continuation; `n` = first n continuation pages only; `false` = disable.
- New `continued-caption` parameter (bool, default `false`) — adds a "Cont. Table X.Y caption" header on each continuation page (same format as `refer-to` mode; number anchored to the main table). The caption row and markdown header row live in two separate `table.header` blocks (with `level: 1/2`), so `continued-caption: true` can coexist with `repeat-header: false`. Continuation cells locate the main table via `query(label)` and delegate to the refer-to branch of `_make_caption_content`, so they never re-register a figure or advance the counter; cap-able auto-synthesises a hidden label when the user did not provide one.
- `bicap` now supports the `#bicap(...)[body]` calling form: wraps caption + body in a single non-breakable block, conceptually a cap-able-flavoured `figure(body, caption: ...)`. `body` can also be passed via the named arg `body: [...]`.
- `bicap` gains `continued-caption` (bool, default false): when `kind: "table"`, every native `#table()` in body has its caption repeated on each continuation page (same mechanism as `captab(continued-caption: true)`). Convention: one table per bicap; multiple tables share the same injected caption.
- `bicap` gains `repeat-header` (auto/bool/int, default auto): overrides the `repeat` setting of the markdown header inside body's `#table()`. Independent of `continued-caption`; e.g. `repeat-header: false` to disable header repetition while keeping caption repetition (or alone).
- `captab` and `captab-style` gain `three-line-table` (bool, default `true`): set to `false` to drop the three-line style and use Typst's default `table()` grid stroke instead. `top-rule` / `middle-rule` / `bottom-rule` are inactive in that mode; user `hlines` / `vlines` still work.
- `captab` adds an explicit `align: auto` parameter and a `..extra-args` sink that forwards any other named arguments (`fill` / `stroke` / `gutter` / `column-gutter` / `row-gutter` / `rows`, etc.) straight to the underlying `table(...)`, matching tablem's advanced-usage surface. A user-supplied `stroke` overrides the `stroke: none` we emit in three-line mode (combine

with `three-line-table: false` for a clean grid; otherwise the three manual hlines stack on top of your stroke).

- `cell-inset` and `inset` are now aliases of each other across `captab` / `captab-style`: previously `captab` only accepted `inset:` while the global config only accepted `cell-inset:`. Both names now work everywhere; if both are passed, `cell-inset` wins (matches the canonical state key).
- Table-width system reworked. Default changed from "fill 100% of text width" to `auto` (sized to content). New `width` parameter on `captab` and `captab-style`, accepting `auto` / `<length>` (e.g. `8cm`) / `<ratio>` (e.g. `80%`, `80.5%`). `set-table-width` gains a `width:` parameter (legacy `percentage:` int still supported and converted to `<int>%`). Priority: per-call > global state > `auto`. When `width != auto` and the user did not pass `columns`, the default switches to `(1fr,) * N` so the table fills the configured width; when `width == auto`, the default is `(auto,) * N` for content sizing.
- Fix `captab`'s alignment handling: passing an already-2D alignment (e.g. `align: horizon + center`) no longer panics with "cannot add a vertical and a 2D alignment". The logic now only appends `+ horizon` when the alignment's `y` component is missing.
- New `caption-position` config (`top` / `bottom`) — controls whether the caption is above or below body for both `captab` and `bicap()[body]`. Defaults follow kind: table=top, figure=bottom. Override globally via `captab-style` / `capfig-style` / `cap-style`, or per call via `captab(caption-position: ...)` / `bicap(position: ...)`. Known limitation: `caption-position: bottom` and `continued-caption: true` are incompatible (continuation repetition would require `table.footer`, which locks the caption inside the table's column boundaries); when both are set, `continued-caption` is silently disabled.
- New `caption-align` config (default `"center"`) — controls horizontal alignment of the caption. Five values: `"center"` / `"left"` / `"right"` (table-local) / `"text-left"` / `"text-right"` (text-area-local). Also accepts a dict `(main: ..., continued: ...)` to split main vs continuation captions (missing keys fall back to `"center"`). Exposed via `cap-style` / `captab-style` / `capfig-style` / `captab` / `bicap`. Known limitation: continuation captions live inside `table.header` and are locked to the table's column width — `"text-left"` / `"text-right"` on the continuation side silently degrade to `"left"` / `"right"`; for true text-anchored continuation captions, fall back to manual `refer-to` splitting.
- Renamed `repeat-caption` → `continued-caption` (no alias retained, since 0.1.0 has not been merged yet). Affects both `captab` and `bicap`. The semantics are unchanged — bool, default `false`, controls whether the caption is auto-repeated on each continuation page.
- Fixed `numbering-format` being ignored when `use-chapter: false` — that branch hard-coded `numbering("1", num)`, so user-set formats like `"(A)"` or `"App. 1"` had no effect. Both branches now honour the configured format.
- New `placement` config (default `none`) — mirrors Typst's native `figure(placement: ...)`. Four values: `none` (in-flow, default) / `top` / `bottom` (float to top/bottom of current or next page) / `auto` (Typst picks `top` or `bottom` based on which edge is closer to the current position). Implemented by wrapping the outer block in `place(float: true)`;

the hidden figure (cross-ref anchor) rides along inside the float wrapper, so `@ref` resolves to the float's landing page. Exposed via `cap-style` / `captab-style` / `capfig-style` / `captab` / `capfig` / `capsubfig` / `bicap`. Known limitations: (1) floats force `breakable: false` (Typst doesn't allow floats to break across pages); (2) tables taller than a page will overflow when floated — keep `placement: none` for them; (3) incompatible with `continued-caption: true` (which depends on page breaks).

- New `caption-text` config (default `(:)`) — pass-through dict for the caption's `text(...)` rendering. Accepts any Typst native `text()` parameter (`font` / `fill` / `tracking` / `spacing` / `stretch` / ...). Two forms: a flat dict (`(font: "Times")`) applies to the whole caption; a nested dict (containing any of `whole` / `prefix` / `supplement` / `number` / `body`) layers per part with inner overriding outer. Solves the "show rule on figure.caption doesn't change the font" problem — cap-able's caption isn't a real `figure.caption` element (it's manually rendered to support bilingual layout), so font/fill/tracking changes must go through `caption-text`. Exposed on `cap-style` / `captab-style` / `capfig-style`; applies to continuation captions as well.
- `numbering-format` now accepts `str | function`, matching the first argument of Typst's native `numbering()`. The function form receives all heading levels + the figure/table number (when `use-chapter: true`) or just the figure/table number (when `use-chapter: false`); the user function decides how to consume them. `calculate-chapter-levels` returns 0 for function-form formats (chapter-level is meaningless there). Also switched the format-string scan to grapheme clusters (`.clusters()`) so multi-byte literals like `"附 1"` no longer panic on UTF-8 boundaries.
- `bicap` gains a `breakable` parameter (bool, default `true`) — whether the outer block may break across pages. The previous design locked caption + body inside `breakable: false` and prevented an inherently breakable body (e.g. a long captab) from flowing across pages; the new default is transparent. Pass `breakable: false` explicitly to restore the old "atomic caption + body" behaviour.
- Subfigure `label-style-override` dict expanded with four new keys: `style` / `font` / `size` / `offset`, allowing per-subfig override of the overlay label's format string, font, size and offset. `style` also propagates to cross-references — when ref follows overlay (subcaption hidden), `@ref` renders that subfig's own format so the letter matches what's stamped on the figure. The subcaption side does not participate in per-subfig overrides (to keep subcaptions visually uniform).
- Subfigure `label-style` now accepts `str | dict` — passing a string (e.g. `"(a)"`) keeps overlay label and subcaption prefix coupled (default behaviour); passing a dict `(overlay: "a)", subcaption: "(a)")` decouples them, with missing keys falling back to the package default `"(a)"`. Cross-reference letters follow whichever side is actually visible — subcaption when shown (with `show-subcaption-label: true`), else overlay (when `label-mode: "overlay"`), with subcaption as the defensive fallback when neither is visible — so the rendered `@ref` letter always matches what the reader sees.

- New `subcaption-number-title-spacing` config (default `auto`) — controls the gap between the subcaption number prefix and its body. `auto` inherits the main caption's `number-title-spacing` so subcaptions render with the same separator as the main caption (instead of the previous bare single space). Configure globally via `capfig-style(subcaption-number-title-spacing: ...)` or per call via `capsubfig(subcaption-number-title-spacing: ...)`.

Compatibility:

- Default change: `breakable` defaults to `true` from 0.1.0 onward (versus the implicit `false` of 0.0.x). To preserve the old atomic behavior, pass `breakable: false` explicitly or set it globally via `captab-style.with(...)`.
- Default change: table width defaults to `auto` (content-sized) instead of "fill 100% of text width". To restore the old behaviour, pass `set-table-width(width: 100%)` or `captab-style.with(width: 100%)`. Tables that explicitly use `fr` columns continue to expand by their fr factors (constrained by the parent block); tables that previously relied on the implicit 100% width with no `columns` will now appear content-sized.

XI.2 Version 0.0.2

Bug fixes:

- `captab-style` / `capfig-style` / `cap-style` switched to patch semantics. Every parameter now defaults to `auto`, and only the fields you explicitly pass get written back to state. This means you can call `captab-style.with(...)` repeatedly to override one dimension at a time (e.g. enabling `outline-bilingual`) without resetting omitted fields (e.g. `cell-inset`) to their function defaults. Show / set rules now read the latest merged values via `context { state.get() }` at render time.

New features:

- `captab` and `captab-style` gain `top-rule` / `middle-rule` / `bottom-rule` parameters for customising the three-line table's top, middle, and bottom rules. Accept any Typst stroke value, e.g. `2pt + red`, `stroke(thickness: 1pt, dash: "dashed")`. Defaults remain `1.5pt` / `0.5pt` / `1.5pt`.
- `captab` gains a `columns` parameter as the preferred alias of `cols`, matching Typst's native `table(columns: ...)` naming. `cols` is still accepted as a back-compat alias.

Documentation updates:

- Complete parameter tables include `top-rule` / `middle-rule` / `bottom-rule` / `columns`.
- Added patch-semantics callouts before `captab-style` and `capfig-style` sections.
- Added `== Three-Line Strokes` subsection with usage examples.
- All `cols:` examples rewritten as `columns:`.

XI.3 Version 0.0.1

Initial release with:

- `captab` three-line table support
- Bilingual caption support
- Continued tables and figures
- `capsubfig` subfigure layouts
- Table and figure notes
- 25+ language localization
- RTL language support (Arabic, Hebrew, Persian, Urdu)
- Comprehensive configuration system
- Configuration functions: `cap-style` (unified), `captab-style`, `capfig-style`, `set-table-width`

Part XII
Index